Digital Circuit

沈威宇

November 1, 2026

Contents

Chapter 1 Digital Circuit 1

1 Binary Number 1

I Introduction 1

II Signed Number 1

i Sign and magnitude 1

ii One's complement 1

iii Two's complement 4

III Binary Encoding 5

i Commonly used code 5

ii Binary encodings 6

iii Binary code 6

iv Weighted code 6

v Excess-n code 6

vi Binary-coded decimal (BCD) or 8-4-2-1 code 6

vii Gray adjacency 6

viii Reflected binary code (RBC), reflected binary (RB), or (reflecting) gray code 6

ix One-hot and one-cold encoding 7

x 2-out-of-5 code 7

xi Character Encoding 7

xii Parity bit or check bit 7

2 Combinational Circuit 7

I Digital circuit and boolean algebra 7

II Logic gate 8
i Logic gate symbols 8
ii Alternative symbols 9
iii Fan-ins 9
iv Fan-out 10
v Buffer 10
III Digital circuit 10
i Digital system 10
ii Analog system 10
iii Circuit (ckt) 10
iv Digital circuit or switching circuit 10
v Logic circuit 10
vi System design 10
vii (Digital) logic design 11
viii Circuit design 11
ix Combinational circuit 11
x Sequential circuit 11
xi Switch, switching device, or switch element 11
xii Active high and low 11
xiii Bus 11
xiv Iterative Circuits 11
xv Bits significances 11
xvi Pull-up and Pull-down resistors 12
xvii Feedback 12
xviii Serial and Parallel Input and Output 12
xix Transmission Gate (TG), Analog Switch, or Bilateral Switch 12
IV Multi-level combinational circuits 13
i Introduction 13
ii Design of circuits of AND and OR gates 13
iii Degeneracy 13
iv Design of two-level circuits 14

v Design of multi-level NAND- and NOR-gate circuits . . . 15
vi Circuit conversion using inversion bubbles . . . 15
V Multiple-output combinational circuits . . . 16
i Minimal cost realization . . . 16
ii Karnaugh Maps . . . 17
iii Design of multiple-output NAND- and NOR-gate circuits . . . 17
VI Delays and Hazards . . . 18
i Propagation delays and timing diagram . . . 18
ii Inertial delays . . . 18
iii Hazards, Glithces, or Spikes . . . 18
VII Simulation and Testing . . . 20
i Verilog, SystemVerilog, and VHDL . . . 20
ii Four-valued logic . . . 20
iii Simulation and testing . . . 21
VIII Integrated Circuit (IC) . . . 21
i Integrated Circuit (IC), microcip, or chip . . . 21
ii Integrated circuit package . . . 21
iii Scales . . . 22
IX Combinational circuits . . . 22
i Multiplexer (MUX) . . . 22
ii Three-state buffer or tri-state buffer . . . 23
iii Bidirectional I/O pin . . . 23
iv Half adder (HA) . . . 24
v Half subtractor (HS) . . . 24
vi Full adder (FA) . . . 24
vii Full subtractor (FS) . . . 25
viii Parallel (binary) adder, ripple-carry adder (RCA), or carry propagate adder (CPA) . . . 26
ix Parallel (binary) subtractor, ripple-borrow subtractor (RBS), or borrow propagate subtractor (BPS) . . . 26
x Adder-subtractor . . . 27

xi One's complement adder . . . 27
xii One's complement subtractor . . . 27
xiii Carry-lookahead adder (CLA) . . . 27
xiv Carry-save adder (CSA) . . . 28
xv Carry-select adder (CSLA) . . . 29
xvi Carry-skip adder (CSkA) or carry-bypass adder . . . 30
xvii Parallel prefix adder (PPA) . . . 30
xviii Kogge–Stone adder (KSA or KS) . . . 31
xix Brent–Kung adder (BKA or BK) . . . 31
xx Han–Carlson adder (HCA) . . . 31
xxi Sklansky adder (SA) or divide-or-conquer adder . . . 31
xxii Decoders . . . 32
xxiii Normal encoders . . . 32
xxiv Priority encoders . . . 32
xxv Comparator . . . 33
xxvi Lookup table (LUT) . . . 33
X Devices . . . 33
i Read-only memory (ROM) . . . 33
ii Programmable Logic Devices (PLDs) . . . 34
iii Programmable Logic Arrays (PLAs) . . . 35
iv Programmable Array Logic (PAL) . . . 36
v Complex Programmable Logic Devices (CPLDs) . . . 36
vi Field-Programmable Gate Arrays (FGPAs) . . . 36
3 Sequential Circuit . . . 37
I Introduction . . . 37
i Clock (CLK) signal . . . 37
ii Synchronous and asynchronous circuit . . . 37
iii Latch . . . 37
iv Asynchronous, Edge-triggered, and Level-triggered latch . . . 37
v Gated or level-sensitive latch . . . 37
vi (Edge-triggered) Flip-flop (FF) . . . 38

vii Metastable state 38
viii Latch response time or delay time 38
ix Flip-flop propagation delay, setup time, and hold time 38
x State variable 39
xi Present state and next state 39
xii Next-state Equation or Characteristic Equation 39
xiii Essential hazard 39
xiv Race condition, race hazard, or race 39
xv Cross-Coupled Gates 39
xvi Logic-equivalent circuit using standard latches and flip-flops. 40
II Latches 40
i (Active-high) S-R Latch 40
ii Active-low S-R Latch 41
iii Switch Debouncing with an S-R Latch 42
iv Gated S-R Latch 43
v (Gated) D Latch 44
vi (Asynchronous) J-K Latch 45
vii Gated J-K Latch 45
III Flip-flops 46
i Edge-triggered flip-flops 46
ii (Edge-triggered) S-R flip-flop (SRFF or SR-FF) 46
iii (Edge-triggered) D flip-flop (DFF or D-FF) 46
iv (Edge-triggered) J-K flip-flop (JKFF or JK-FF) 46
v (Edge-triggered) T flip-flop (TFF or T-FF) 46
vi Pulse-triggered flip-flops. 47
vii Master-slave flip-flops. 47
viii Master-slave S-R flip-flop 47
ix Master-slave D flip-flop 48
x Master-slave J-K flip-flop 48
xi Flip-flops with asynchronous clear and preset 48
xii Flip-flops with clock enable or chip enable 48

xiii Sequential parity checker 49
IV Analysis and Design of Sequential Circuits 49
i Analysis by signal tracing 49
ii As automata. 50
iii Glitch of Mealy Circuit 50
iv Mealy and Moore implementation 50
v State table and (state) transition table 51
vi State/transition/state transition diagram/graph for Moore circuit 51
vii State/transition/state transition diagram/graph for Mealy circuit 51
viii Incompletely specified state. 51
ix Alphanumeric state graph notation 52
x Counter. 52
xi State/transition/state transition diagram/graph/cycle 52
xii Next-state Map 52
xiii Excitation table 53
xiv Flip-flop input (K-)map 53
xv D flip-flop excitation table 53
xvi T flip-flop excitation table 53
xvii S-R flip-flop excitation table. 53
xviii J-K flip-flop excitation table. 54
xix D input map 54
xx T input map 54
xxi S and R input map 54
xxii J and K input map 54
xxiii Output functions and output maps 54
xxiv Analysis by state table or transition table 55
xxv Clock skew 55
xxvi General model for Mealy circuits 55
xxvii General model for Moore circuits 55
xxviii General model for synchronous circuits without outputs 56
xxix Timing constraints 56

xxx Output delay 57
V Register 57
i Register 57
ii Data Transfer Between Registers with Tri-state Buffers 57
iii Data Transfer Between Registers with Multiplexers. 57
VI Shift Register 57
i Shift Register 57
ii Serial-In Serial-Out (SISO) Shift Register 58
iii Serial-In Parallel-Out (SIPO) Shift Register 58
iv Parallel-In Serial-Out (PISO) Shift Register 58
v Parallel-In Parallel-Out (PIPO) Shift Register 58
vi Parallel-In Serial-Out (PISO) Bidirectional Shift Register 59
vii Serial-In Serial-Out (SISO) Bidirectional Shift Register 59
viii Parallel-In Parallel-Out (PIPO) Bidirectional Shift Register 59
ix Serial-In Parallel-Out (SIPO) Bidirectional Shift Register 60
x Synchronizer 60
VII Accumulator and processing unit 60
i Accumulator 60
ii Parallel processing unit. 60
iii Parallel adder with accumulator 60
iv Serial processing unit 61
v Serial adder with accumulator 61
VIII Synchronous Counter 62
i Synchronous Counter. 62
ii Shift Register Counter 62
iii Ring Counter 62
iv Johnson counter, Twisted Ring Counter, or Mod-2n Counter 62
v Binary Up Counter 62
vi Down Counter 63
vii Up/Down (U/D) Counter 63
viii Loadable Counter 64

ix Design of General Synchronous Counter with T Flip-Flops 64
x Design of General Synchronous Counter with D Flip-Flops 64
xi Design of General Synchronous Counter with S-R Flip-Flops 65
xii Design of General Synchronous Counter with J-K Flip-Flops 65
IX Linear Feedback Shift Register (LFSR) 65
X Asynchronous counter or ripple counter 66
i Asynchronous counter or ripple counter 66
ii Asynchronous down and up counter or frequency divider 66
XI Sequence Detector 66
i Definition 66
ii Design with SIPO shift register 67
iii Design as finite-state machine (FSM) performing Knuth–Morris–Pratt (KMP) algorithm 68
iv Trie 70
v Sequence detector detecting multiple sequences 70
XII Line code converter 71
i Line code 71
ii Clock recovery 71
iii DC balance 71
iv NRZ (non-return-to-zero) code 71
v NRZI (non-return-to-zero-inverted) code 71
vi RZ (return-to-zero) code 71
vii Manchester code 71
viii Mealy NRZ to NRZI and NRZI to NRZ converter 72
ix Moore NRZ to NRZI and NRZI to NRZ converter 72
x Mealy NRZ to RZ converter 72
xi Moore NRZ to RZ converter 72
xii Moore RZ to NRZ converter 72
xiii Mealy NRZ to Manchester converter 72
xiv Moore NRZ to Manchester converter 73

XIII Elimination of Redundant States . . . 74
i Reachability . . . 74
ii Distinguishable . . . 74
iii Equivalent states . . . 74
iv Implication chart or implication table . . . 75
v Compatible states . . . 75
vi Merger table or merger chart . . . 76
vii Equivalent deterministic finite-state transducers . . . 77
viii Finding equivalent deterministic finite-state transducers with implication chart or pair chart . . . 77
ix Conversion between Moore and Mealy machine . . . 78
XIV State Assignment . . . 78
i Cost . . . 78
ii Equivalent state assignments . . . 78
iii Interchanging digits . . . 78
iv Symmetrical flip-flop . . . 79
v Trial-and-error method . . . 79
vi Assignment map method . . . 79
vii One-hot and one-cold state assignment . . . 80
XV Design of Sequential Circuits Using ROMs, PLAs, CPLDs, and FGPAs . . . 80
i Design of Sequential Circuits Using ROMs . . . 80
ii Design of Sequential Circuits Using PLAs . . . 80
iii Design of Sequential Circuits Using CPLDs and FGPAs . . . 81
XVI Simulation and Testing . . . 81
XVII Multiplier . . . 81
i General . . . 81
ii Shift-add multiplier . . . 82
iii Array multiplier . . . 82
iv Wallace (tree/reduction) multiplier . . . 82
v Dadda (tree/reduction) multiplier . . . 82
vi Booth's multiplication algorithm or Booth encoding . . . 83

XVIII Divider 84
i General 84
ii Restoring Divider 84
iii Non-restoring Divider 84
iv SRT Divider 85
XIX Arithmetic Logic Unit (ALU) 86
XX State Machine Chart (SM Chart) 86

Chapter 1 Digital Circuit

1 Binary Number

I Introduction

A binary digit is called a bit, which is either 0 or 1. Binary arithmetic is the same as decimals, except that "invert" or "flip" means converting 0 to 1 and 1 to 0, and "complement" means inverting all bits.

The most significant bit (MSB) or most significant digit is the bit with the highest value place in a binary number and is the leftmost bit in standard binary notation; the least significant bit (LSB) or least significant digit is the bit with the lowest value place in a binary number and is the rightmost bit in standard binary notation.

4 bits is called a nibble; 8 bits is called a byte or an octet of bits.

II Signed Number

Suppose you have N bits to present a signed number. There's three common method:

i Sign and magnitude

The MSB represents sign, in which 0 represents positive and 1 represents negative; the remaining represents magnitude.

It can represent numbers range from $-(2^{N-1}-1)$ to $2^{N-1}-1$. It has two zero ($+0$ and -0).

It might be the easiest to read, but its arithmetic is the hardest to compute.

ii One's complement

The nonnegative numbers are the same as those in sign and magnitude; the negative numbers invert all bits of its absolute value, i.e. (2^N-1) minus its absolute value, that is, if a negative integer B written in 1's complement is

$$B = b_{N-1}b_{N-2}\dots b_1 b_0,$$

then

$$B = -2^{N-1} + 1 + \sum_{i=0}^{N-2} b_i 2^i.$$

It can represent numbers range from $-(2^{N-1}-1)$ to $2^{N-1}-1$. It has two zero ($+0$ and -0).

Its arithmetic is easier than sign and magnitude and harder than 2's complement to compute.

The negation can be computed by inverting all bits, that is, subtracting it from 2^N-1.

The addition can be computed the same as unsigned numbers; however, if the sum had a carry out from the MSB, we must add it back to the LSB, called the "end-around carry (EAC)".

Proof. Assume no overflow conditions.

Suppose we want to add two n-bit numbers A and B, and A and B in 1's complement are A_1 and B_1 (treated as unsigned). Without loss of generality, assume $A \geq B$.

Let:

$$S = A_1 + B_1, \quad R = S \mod 2^N, \quad C = \left\lfloor \frac{S}{2^N} \right\rfloor,$$

and S_c be the correct sum of A and B in 1's complement (treated as unsigned).

Case 0: $A, B \geq 0$

$$S = A_1 + B_1 = A + B = S_c$$

Case 1: $A \geq 0 > B \wedge |B| \leq A$

$$S = A_1 + B_1 = 2^N - 1 + A - |B|$$

$$R = A - |B| - 1, \quad C = 1$$

$$S_c = A - |B| = R + C$$

Case 2: $A \geq 0 > B \wedge |B| \geq A$

$$S = A_1 + B_1 = 2^N - 1 + A - |B| = S_c = R$$

Case 3: $0 \geq A \geq B$

$$S = A_1 + B_1 = 2^{N+1} - 2 - |A| - |B|$$

$$R = 2^N - 2 - |A| - |B|, \quad C = 1$$

$$S_c = 2^N - 1 - |A| - |B| = R + C$$

□

If the sum of two positive numbers is greater than $2^{N-1}-1$, it overflows into the negative range; if the sum of two negative numbers is less than $-2^{N-1}+1$, it overflows into the positive range. This makes overflow detection straightforward.

The subtraction can be computed the same as unsigned numbers; however, if the difference had a borrow out from the MSB, we must subtract it back to the LSB, called the "end-around borrow (EAB)".

Proof. Assume no overflow conditions.

Suppose we want to subtract a n-bit number B from another n-bit number A, and A and B in 1's complement are A_1 and B_1 (treated as unsigned).

Let:

$$S = A_1 - B_1, \quad R = S \mod 2^N, \quad C = \left\lfloor \frac{S}{2^N} \right\rfloor,$$

and S_c be the correct sum of A and B in 1's complement (treated as unsigned).

Case 0: $A \geq B \geq 0$

$$S = A_1 - B_1 = A - B = S_c$$

Case 1: $B \geq A \geq 0$

$$S = A_1 - B_1 = 2^N - 1 - (B - A)$$

$$R = B - A + 1, \quad C = 1$$

$$S_c = B - A = R - C$$

Case 2: $A \geq 0 > B$

$$S = A_1 - B_1 = A - \left(2^N - 1 - |B|\right)$$

$$R = A + |B| + 1, \quad C = 1$$

$$S_c = A + |B| = R - C$$

Case 3: $B \geq 0 > A$

$$S = A_1 - B_1 = 2^N - 1 - |A| - B = R = S_c$$

Case 4: $0 > A \geq B$

$$S = A_1 - B_1 = 2^N - 1 - |A| - \left(2^N - 1 - |B|\right) = |B| - |A| = R = S_c$$

Case 5: $0 > B > A$

$$S = A_1 - B_1 = 2^N - 1 - \left(2^N - 1 - |A| - \left(2^N - 1 - |B|\right)\right) = 2^N - 1 + |A| - |B|$$

$$R = |A| - |B| + 1, \quad C = 1$$

$$S_c = |A| - |B| = R - C$$

□

If the difference of a positive number and a negative number is greater than $2^{N-1} - 1$, it overflows into the negative range; if the difference of a negative number and a positive number is less than $-2^{N-1} + 1$, it overflows into the positive range. This makes overflow detection straightforward.

iii Two's complement

The nonnegative numbers are the same as those in sign and magnitude; the negative numbers invert all bits of its absolute value and add 1, i.e. 2^N minus its absolute value, where -2^{N-1} is only MSB being 1 and others being 0, that is, if a negative integer B written in 1's complement is

$$B = b_{N-1}b_{N-2}\dots b_1b_0,$$

then

$$B = -2^{N-1} + \sum_{i=0}^{N-2} b_i 2^i.$$

Proof.

$$\begin{aligned} B = -|B| &= -\left(\sum_{i=0}^{n-2}(1-b_i)+1\right) \\ &= -\frac{2^{n-1}-1}{2-1} - 1 + \sum_{i=0}^{n-2} b_i 2^i \\ &= -2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i \end{aligned}$$

□

It can represent numbers range from -2^{N-1} to $2^{N-1}-1$. It has only one zero.

Its arithmetic is the easiest to compute. It is the most commonly used representation for signed numbers in computers.

The negation can be computed by inverting all bits and add 1, that is, subtracting it from 2^N.

Proof. Let negative integer B written in 2's complement be

$$B = b_{n-1}b_{n-2}\dots b_1b_0,$$

that is,

$$B = -2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i.$$

The negation of it, which is positive, is

$$\begin{aligned} -B &= 2^{n-1} - \sum_{i=0}^{n-2} b_i 2^i \\ &= 2^{n-1} + \sum_{i=0}^{n-2}(1-b_i)2^i - \sum_{i=0}^{n-2} 2^i \\ &= 2^{n-1} + \sum_{i=0}^{n-2}(1-b_i)2^i - \frac{2^{n-1}-1}{2-1} \\ &= \sum_{i=0}^{n-2}(1-b_i)2^i + 1 \end{aligned}$$

□

The addition can be computed the same as unsigned numbers.

If the sum of two positive numbers is greater than $2^{N-1}-1$, it overflows into the negative range; if the sum of two negative numbers is less than -2^{N-1}, it overflows into the positive range. This makes overflow detection straightforward.

The subtraction can be computed the same as unsigned numbers.

If the difference of a positive number and a negative number is greater than $2^{N-1}-1$, it overflows into the negative range; if the difference of a negative number and a positive number is less than -2^{N-1}, it overflows into the positive range. This makes overflow detection straightforward.

III Binary Encoding

i Commonly used code

Decimal/Hexadecimal Digit	Binary Code	6-3-1-1 Code	Excess-3 Code	2-out-of-5 Code	Reflec
0	0000	0000	0011	00011	
1	0001	0001	0100	00101	
2	0010	0011	0101	00110	
3	0011	0100	0110	01001	
4	0100	0101	0111	01010	
5	0101	0111	1000	01100	
6	0110	1000	1001	10001	
7	0111	1001	1010	10010	
8	1000	1011	1011	10100	
9	1001	1100	1100	11000	
A	1010				
B	1011				
C	1100				
D	1101				
E	1110				
F	1111				

ii Binary encodings

Binary encodings are encodings that encode things into binary numbers.

iii Binary code

Binary code is an encoding of a sequence of things that encode the ith one to binary number i.

iv Weighted code

A k-bit ($k \geq 4$) weighted code of numbers has the property that if the weights are integers $w_{k-1}, w_{k-2}, \ldots w_0$ where for all $0 \leq i < j < k$, $w_i < w_j$, the code $a_{k-1}a_{k-2}\ldots a_0$, where $a_i \in \{0, 1\}$, represents number N, then

$$N = \sum_{n=0}^{k-1} w_n a_n.$$

If a number N has more than one possible codes in a specific set of weights, the one such that the unsigned binary number presented by the code is least is usually used.

v Excess-n code

Excess-n encoding is an encoding of a sequence of things that encode the ith one to binary number $i + n$.

Excess-3 code of decimal digits is sefl-complementing, i.e. the complement of it is the 9's complement of the number.

vi Binary-coded decimal (BCD) or 8-4-2-1 code

Binary-coded decimal (BCD) or 8-4-2-1 code, is an encoding of decimal digits (0 to 9) in binary, in which each decimal digit is encoded separately using 4 bits to its binary form, which makes it easy to convert between decimal numbers and binary.

vii Gray adjacency

Two binary numbers in a binary encoding are called (gray) adjacent iff they differ only in one bit. An encoding of a sequence of things is called gray-like if each step changes only one bit, which can be useful to minimize transition errors.

viii Reflected binary code (RBC), reflected binary (RB), or (reflecting) gray code

Let the n-bit gray code be denoted as G_n for all $n \in \mathbb{N}_0$, and integer $0 \leq m \leq 2^n - 1$ in G_n be denoted as $G_n(m)$. The gray code is defined as follows:

$$G_0(0) = \varepsilon,$$

$$\forall n \in \mathbb{N} : \forall m \in \mathbb{N}_0 \wedge m < 2^{n-1} : G_n(m) = 0G_{n-1}(m),$$

$$\forall n \in \mathbb{N} : \forall m \in \mathbb{N}_0 \wedge m \geq 2^{n-1} : G_n(m) = 1G_{n-1}\left(m - 2^{n-1}\right),$$

where ε denotes the zero-bit code in G_0, and juxtaposition denotes concatenation.

ix One-hot and one-cold encoding

A one-hot (encoding) is an encoding where each code is a unique binary sequence with only one 1 bit.

A one-cold (encoding) is an encoding where each code is a unique binary sequence with only one 0 bit.

One-hot and one-cold encoding always change exactly two bits when changing state, errors can be detected by checking parity, and detecting states costs less, but requires more bits.

x 2-out-of-5 code

Every code in 2-out-of-5 code always has exactly two 1 bits.

xi Character Encoding

Many applications of computers require the processing of data which contains numbers, letters, and other symbols. In order to transmit such data to or from a computer or store it internally in a computer, each character must be represented by a binary code.

Common character encodings include:

- **ASCII code (American Standard Code for Information Interchange)**: A 7-bit code, so 128 different code combinations are available.
- **Unicode or the Unicode Standard (TUS)**: Includes international characters, symbols, etc. such as those in Arabic, CJK Unified Ideographs, and Emoticons. The Unicode Standard itself defines three encodings: UTF-8, UTF-16, and UTF-32. UTF-8 is the most widely used, in part due to its backwards-compatibility with ASCII.

xii Parity bit or check bit

A parity bit, or check bit, is a bit added to a string of binary code. Parity bits are a simple form of error detecting code. Even parity bit means that the parity bits are added such that the total number of 1-bits in the string is even. Odd parity bit means that the parity bits are added such that the total number of 1-bits in the string is odd.

Parity bits are generally applied to the smallest units of a communication protocol, typically bytes, that is, each byte consists of 7 bits of data bits and 1 parity bit, although they can also be applied separately to an entire message string of bits.

2 Combinational Circuit

I Digital circuit and boolean algebra

Switching devices are essentially two-state devices, and thus can be studied with Boolean algebra (aka switching algebra), e.g. switches which are open (0) or closed (1), transistors with high (1) or low (0) output voltages.

II Logic gate

A logic gate is an electronic device that performs a Boolean function.

i Logic gate symbols

Logic gates symbols (inputs at left, two inputs for example for AND, OR, NAND, NOR, XOR, and XNOR gate):

Type	Distinctive shape (ANSI/IEEE Std 91/91a-1991	Rectangular shape (IEEE Std 91/91a-1991/IEC 60617-12:1997)	Boolean function
(Logic) buffer (gate)			A
NOT gate / inverter			A'
AND gate			AND
OR gate			OR
NAND gate			NAND
NOR gate			NOR
XOR gate			XOR
XNOR gate			XNOR
IMPLY gate			IMPLY
NIMPLY gate			NIMPLY

An empty circle, called inversion bubble or bubble, means inverting the input before inputting to the gate when at a input to a gate, and means inverting the output before outputting from the gate when at the output from a gate.

The diagrams of logic circuits are called block diagrams. The number of gate inputs of a circuit is the sum of the number of inputs to all gates in the circuit.

ii Alternative symbols

By using inversion bubbles at the inputs instead of the output in logic gates symbol except IMPLY and NIMPLY gates, using inversion bubbles at the output and second input instead of the first input in IMPLY gate symbol, and using inversion bubbles at the second input instead of the first input and the output in NIMPLY gate symbol, we obtain the alternative gate symbols (inputs at left, two inputs for example for AND, OR, NAND, NOR, XOR, and XNOR gate):

Type	Distinctive shape (ANSI/IEEE Std 91/91a-1991	Rectangular shape (IEEE Std 91/91a-1991/IEC 60617-12:1997)	Boolean function
(Logic) buffer (gate)			A
NOT gate / inverter			A'
AND gate			AND
OR gate			OR
NAND gate			NAND
NOR gate			NOR
XOR gate			XOR
XNOR gate			XNOR
IMPLY gate			IMPLY
NIMPLY gate			NIMPLY

Sometimes multiple (typically two) outputs are drawn on a gate with some of them (typically one of the two) with inversion bubble. This is logically equivalent to drawing one gate of that type for each of the noninverted or inverted outputs with the same input lines.

iii Fan-ins

The maximum or actual number of inputs to a device is called the fan-in or fanin.

An AND, OR, NAND, NOR, XOR, or XNOR gate with n inputs or fan-in n is sometimes called with

suffix n, e.g. NAND2 gate is a NAND gate with 2 inputs or fan-in 2.

An AND, OR, NAND, NOR, XOR, or XNOR gate without fan-in specified is sometimes assumed to have fan-in 2.

The fan-in of a practical logic gate is limited and may be two, three, four, eight, or some other number depending on the type of gates used.

iv Fan-out

The maximum or actual number of inputs driven by a device's output is called the fan-out or fanout.

v Buffer

A gate output can only be connected to a limited number of other device inputs without degrading the performance of a digital system. A simple buffer may be used to increase the driving capability of a gate output.

III Digital circuit

i Digital system

A system that deals with signals that have discrete values.

ii Analog system

Deals with signals that vary continuously over time. The output might have an error depending on the accuracy of the components used.

iii Circuit (ckt)

A complete electrical network, that is, an interconnection of electrical components, with a closed-loop giving a return path for current.

iv Digital circuit or switching circuit

A circuit that deals with input and output signals that have discrete values, or, as typically the case, a logic circuit.

v Logic circuit

A circuit that deals with binary input and output signals, that is, either 0 or 1. Below, all circuits are logic circuit unless otherwise specified.

vi System design

The highest level of the design of systems, where you break the system into subsystems, specify what each subsystem do, and determine the interconnection and control of the subsystems.

vii (Digital) logic design

The middle level of the design of digital systems, where you specify the logic operations inside each subsystem.

viii Circuit design

The lowest level of the design of digital systems, where you specify the electronic components and their interconnection to form the system.

ix Combinational circuit

A switching circuit whose output value depends only on the present input value.

x Sequential circuit

A switching circuit whose output value depends on the present input value and past input values.

xi Switch, switching device, or switch element

A component that opens or closes a circuit.

xii Active high and low

Active high (aka active-high or noninverted) means the input or output signal is noninverted; active low (aka active-low or inverted) means the input or output signal is (logically equivalent to) inverted.

Unless otherwise specified, a type of electronic components with active high and low version of inputs and/or outputs are assumed to be active high inputs and/or outputs.

xiii Bus

Several signals that perform a common function may be grouped together to form a bus, represented with a single heavy line optionally with a diagonal slash through it specifying the number of bits in the bus.

xiv Iterative Circuits

An iterative circuit consists of a number of identical cells interconnected in a regular manner, each cell has external inputs, external outputs, and interconnections with adjacent cells. The regular structure of an iterative circuit makes it easier to fabricate in integrated circuit form than circuits with less regular structures.

xv Bits significances

For circuits with the concept of LSB to MSB, variables of the kth least significant bit are usually labeled with subscript $_{k-1}$. When writing variables of a specific name of all bits in a row, they are

usually listed from MSB to LSB from leftmost to rightmost. For an n-bit circuit, when treating variables of a specific name of all bits in a row listed from MSB to LSB from leftmost to rightmost as a unsigned binary number, decrements and increments are considered to be with mod n afterwards.

xvi Pull-up and Pull-down resistors

In a digital circuit, if an input pin is not connected to anything (floating), it can pick up random electrical noise. To keep it in a definite voltage we can use pull-up or pull-down resistors.

A pull-up resistor is a resistor connected between a signal line and +Vcc to ensure that the signal stays at logic 1 when no active signal drives it low.

A pull-down resistor is a resistor connected between a signal line and ground to ensure that the signal stays at logic 0 when no active signal drives it high.

xvii Feedback

Feedback occurs when the output of a circuit is fed back into its input. Sequential circuits must contain feedback, but not all circuits with feedback are sequential.

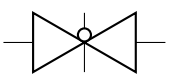
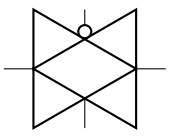
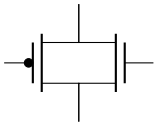
xviii Serial and Parallel Input and Output

Serial input/output means that the data is inputted to/outputted from the circuit sequentially, one bit at a time.

Parallel input/output means that the data is inputted to/outputted from the circuit parallelly, all bits at the same time.

xix Transmission Gate (TG), Analog Switch, or Bilateral Switch

A transmission gate (TG), aka analog switch or bilateral switch, is a CMOS bidirectional switch with two data terminal and a enable signal (aka control signal or select signal) that can either conduct or block the signal path between the two data terminals based on the enable signal. When enable signal is 1, it is closed and can conduct nearly any voltage between ground and supply voltage in both directions; when enable signal is 0, it is open. It consists of a p-channel MOSFET (PMOS), which passes a strong 1 but a weak 0, and a n-channel MOSFET (NMOS), which passes a strong 0 but a weak 1, connected in parallel with the drain and source terminals of the two transistors connected together as the two data terminals and the gate terminals of the PMOS and NMOS driven by the complement of the enable signal and the enable signal respectively.

The symbol of a transmission gate in block diagrams is  or  or 

IV Multi-level combinational circuits

i Introduction

The maximum number of gates cascaded in series between a circuit input and the output is referred to as the number of levels of gates. Thus, a function written in sum-of-products form or in product-of-sums form corresponds directly to a two-level gate circuit. As is usually the case in digital circuits where the gates are driven from flip-flop outputs, we will assume that all variables and their complements are available as circuit inputs. For this reason, we will not normally count inverters which are connected directly to input variables.

If a realization of a circuit requires more inputs to a gate than its fan-in, factoring or multiplying out the logic expression to obtain a more-level realization is necessary.

The level starting with the output gate is numbered 1.

Given multi-input logic gates $A_1, A_2, \dots A_m$, $A_1 - A_2 - \dots A_m i$ circuit means a m-level circuit composed of a level of A_1 gates followed by a level of A_2 gate followed by … an A_m gate at the output.

Given multi-input logic gates $A_1, A_2, \dots A_m$, circuit of $A_1, A_2, \dots A_m$ gates or $A_1, A_2, \dots A_m$-gates circuits implies no particular ordering of the gates.

ii Design of circuits of AND and OR gates

The number of levels in an AND-OR circuit can be increased by factoring the sum-of-products expression from which it was derived. Similarly, the number of levels in an OR-AND circuit can be increased by multiplying out some of the terms in the product-of-sums expression from which it was derived. Sometimes doing so will reduce the required number of gates and gate inputs, and thus reduce the cost of building the circuit. In many application, the number of gates which can be cascaded is limited by gate delays. When the input of a gate is switched, there is a finite time before the output changes. When several gates are cascaded, the time between an input change and the corresponding change in the circuit output may become excessive and slow down the operation of the digital system.

In general, to be sure of obtaining a minimum solution, one must find both the circuit with the AND-gate output and the one with the OR-gate output. If an expression for f' has n levels, the complement of that expression is an n-level expression for f. Therefore, to realize f as an n-level circuit with an AND-gate output, one procedure is first to find an n-level expression for f' with an OR operation at the output level and then complement the expression for f'.

iii Degeneracy

A m-level circuit is degenerate iff it can degenerate into less-than-m-level. A circuit is non-degenerate iff it's not degenerate.

There are 8 degenerate forms, which are two-level circuits:

- AND-AND, which can degenerate into one AND,

- OR-OR, which can degenerate into one OR,
- AND-NAND, which can degenerate into one NAND,
- OR-NOR, which can degenerate into one NOR,
- NAND-NOR, which can degenerate into one AND,
- NOR-NAND, which can degenerate into one OR,
- NAND-OR, which can degenerate into one NAND,
- NOR-AND, which can degenerate into one NOR.

A circuit is degenerate iff any two-level subcircuit of it is of the 8 degenerate form.

A non-deegenerate circuit is an implementation of a sum-of-product-of-sum-…product or product-of-sum-of-product-…sum form.

iv Design of two-level circuits

There are 16 forms of two-level circuit consist of AND, OR, NAND, and NOR gates, 8 of which are the degenerate forms, 4 of which are implementation of SOP form, and 4 of which are implementation of POS form.

The non-degenerate forms that implement SOP form are:

- AND-OR, which can be directly derived from the SOP form,
- NAND-NAND, which can be converted from AND-OR form by replacing all gates with NAND gates and inverting all literals that are inputs to level 1, which can be derived from AND-OR form by taking double negation and applying De Morgan's law on level 1,
- OR-NAND, which can be converted from AND-OR form by replacing AND gates with OR gates and OR gate with NAND gate and inverting all literals, which can be derived from NAND-NAND form by applying De Morgan's law on level 2, or from NOR-OR form by applying De Morgan's law on level 1,
- NOR-OR, which can be converted from AND-OR form by replacing AND gates with NOR gates and inverting all inputs to level 2, which can be derived from AND-OR form by applying De Morgan's law on level 2.

The non-degenerate forms that implement POS form are

- OR-AND, which can be directly derived from the POS form,
- NOR-NOR, which can be converted from OR-AND form by replacing all gates with NOR gates and inverting all literals that are inputs to level 1, which can be derived from OR-AND form by taking double negation and applying De Morgan's law on level 1,
- AND-NOR, which can be converted from OR-AND form by replacing OR gates with AND gates and AND gate with NOR gate and inverting all literals, which can be derived from NOR-NOR form by applying De Morgan's law on level 2, or from NAND-AND form by applying De Morgan's law on level 1,

- NAND-AND, which can be converted from OR-AND form by replacing OR gates with NAND gates and inverting all inputs to level 2, which can be derived from OR-AND form by applying De Morgan's law on level 2.

The procedure of designing a two-level circuit is

1. Simplify the Boolean function to be realized to SOP or POS form.
2. Design an AND-OR or OR-AND circuit.
3. Convert it to the wanted form.

v Design of multi-level NAND- and NOR-gate circuits

The procedure of designing a multi-level NAND-gate circuit is

1. Simplify the Boolean function to be realized to the wanted sum-of-product-of-sum-…product form.
2. Design an AND-OR-AND-…OR or OR-AND-OR…AND-OR circuit.
3. Replace all gates with NAND gates and invert all literals that are inputs to levels of odd numbers.

The procedure of designing a multi-level NOR-gate circuit is

1. Simplify the Boolean function to be realized to the wanted product-of-sum-of-product-…sum form.
2. Design an OR-AND-OR…AND or AND-OR-AND-…OR-AND circuit.
3. Replace all gates with NOR gates and invert all literals that are inputs to levels of odd numbers.

vi Circuit conversion using inversion bubbles

Adding or cancelling inversion bubbles to both ends of any interconnection does not change the Boolean function realized by the circuit. Adding or cancelling even number of inversion bubbles at an input or output does not change the Boolean function realized by the circuit.

Therefore, the gate type at levels i and $i+1$ can be changed by inserting inversion bubbles on both sides of all connections between levels $i+1$ and i. After this operation, the gates at level $i+1$ may appear with double inversion bubbles at output and can be cancelled together, the gates at level i will appear in their alternate symbols (with input bubbles) and can be changed to the standard form. The changing of level $i+1$ after this operation is

Original gate	New gate
Buffer gate	Not gate
Not gate	Buffer gate

Original gate	New gate
AND gate	NAND gate
OR gate	NOR gate
NAND gate	AND gate
NOR gate	OR gate
XOR gate	XNOR gate
XNOR gate	XOR gate

The changing of level i after this operation is

Original gate	New gate
Buffer gate	Not gate
Not gate	Buffer gate
AND gate	NOR gate
OR gate	NAND gate
NAND gate	OR gate
NOR gate	AND gate
XOR gate	XNOR gate
XNOR gate	XOR gate

V Multiple-output combinational circuits

i Minimal cost realization

Solution of digital design problems often requires the realization of several completely or incompletely specified Boolean functions of the same variables, that is, a completely or incompletely specified Boolean vector function. Although each function could be realized separately, the use of some gates in common between two or more functions sometimes leads to a more economical realization. Thus in realizing multiple-output circuits, the use of a minimum SOP for each function does not necessarily lead to a minimum cost realization for the circuit as a whole.

When designing multiple-output circuits, you should try to minimize the total number of gates required. If several solutions require the same number of gates, the one with the minimum number of gate inputs should be chosen.

ii Karnaugh Maps

1. Draw K-map for each output.
2. Group 1's in the same definition in K-maps for single Boolean function; however the collection of groups is the collection of all groups on all maps, in which groups including same cells on different maps are the same and corresponding to only one element in the collection.
3. An allowed collection of groups must follow the following rules:
    - Any 1 must be in at least one group.
    - If any 1 is only covered by one possible group, then that group must be choose. The product represented by the group is called an essential prime implicant of the Boolean vector function.
    - If any group g is completely covered by another group, g must not be chosen. The product represented by a group that is allowed to be chosen according to this rule is called an prime implicant of the Boolean vector function.
4. Find the allowed collections of groups that contain the fewest groups. For each one of them, the sum of the products corresponding to the groups in it derives a minimal-gate SOP realization of the Boolean vector function.
5. Among the realization(s), find the one with fewest literal inputs, that is the minimum cost realization of the Boolean vector function.

iii Design of multiple-output NAND- and NOR-gate circuits

The procedure of designing a multiple-output NAND-gate circuit is

1. Simplify the Boolean vector function to be realized to the wanted sum-of-product-of-sum-…product form.
2. Design an AND-OR-AND-…OR or OR-AND-OR…AND-OR circuit.
3. Replace all gates with NAND gates and invert all literals that are inputs to levels of odd numbers.

The procedure of designing a multiple-output NOR-gate circuit is

1. Simplify the Boolean vector function to be realized to the wanted product-of-sum-of-product-…sum form.
2. Design an OR-AND-OR…AND or AND-OR-AND-…OR-AND circuit.
3. Replace all gates with NOR gates and invert all literals that are inputs to levels of odd numbers.

VI Delays and Hazards

i Propagation delays and timing diagram

When the input to a logic gate is changed, the output will not change instantaneously but change after a finite delay. If the change in output is delayed by time ε with respect to the input, we say that this gate has a propagation delay of ε. In practice, the propagation delay for a 0 to 1 output change may be different than the delay for a 1 to 0 change. Propagation delays for integrated circuit gates may be as short as a few nanoseconds, and in many cases these delays can be neglected. However, in the analysis of some types of sequential circuits, even short delays may be important.

Timing diagram, or timing chart, shows various signals in the circuit as a function of time, often in waveform with time being the horizontal axis and each vertical block plots one signal with the same time scale. The vertical edges on the diagram where the signals change from 0 to 1 are called rising edges or positive edges; the vertical edges on the diagram where the signals change from 1 to 0 are called falling edges or negative edges.

The propagation delay of a digital circuit is the time it needs to stabilize.

The propagation delay of a path of a combinational circuit from one of the inputs to one of the outputs is the sum of the propagation delays of the components on the path. The critical path of a combinational circuit is the path from one of the inputs to one of the outputs that has the largest propagation delay.

ii Inertial delays

If a logic gate will not respond to an input change that is shorter than a certain minimum pulse width ε, we say that this gate has a inertial delay of ε. Quite often the inertial delay value is assumed to be the same as the propagation delay of the gate. In contrast, if a gate always responds to input changes (with a propagation delay), no matter how closely spaced the input changes may be, the gate is said to have an ideal or transport delay.

iii Hazards, Glithces, or Spikes

The behavior that an output goes 0 temporarily when it should stay at 1 or goes 1 temporarily when it should stay 0 due to delay is called a hazard, glitch, or spike.

When the input to a combinational circuit changes, unwanted switching transients may appear in the output. These transients occur when different paths from input to output have different propagation delays. If, in response to any single input change and for some combination of propagation delays, a circuit output may momentarily go to 0 when it should remain a constant 1, we say that the circuit has a (static) 1-hazard (or static-1 hazard). Similarly, if the output may momentarily go to 1 when it should remain a 0, we say that the circuit has a (static) 0-hazard (or static-0 hazard). If, when the output is supposed to change from 0 to 1 (or 1 to 0), the output may change three or more times, we say that the circuit has a dynamic hazard. In each case the steady-state output of the circuit is correct, but a switching transient appears at the circuit

output when the input is changed.

Hazards in a two-level circuit:

- A two-level circuit realizing a SOP form has no static-0 or dynamic hazard. Static-1 hazards in a two-level circuit realizing a SOP form can be detected using a Karnaugh map, in which if any two adjacent 1's are not covered by a same group corresponding to a product term, a 1-hazard exists for the transition between the two 1's, which corresponds to an eliminated consensus term. For an n-variable map, this transition occurs when one variable changes and the other $n-1$ variables are held constant. A such static-1 hazard is denoted as the minterms corresponding the two adjacent 1's with $ beteen them, e.g. 0100$0101. We can eliminate a static-1 hazard by adding a group (product term) that covers the two 1's.
- A two-level circuit realizing a POS form has no static-1 or dynamic hazard. Static-0 hazards in a two-level circuit realizing a POS form can be detected using a Karnaugh map, in which if any two adjacent 0's are not covered by a same group corresponding to a sum term, a 0-hazard exists for the transition between the two 0's, which corresponds to an eliminated consensus term. For an n-variable map, this transition occurs when one variable changes and the other $n-1$ variables are held constant. A such static-0 hazard is denoted as the minterms corresponding the two adjacent 0's with $ beteen them, e.g. 0100$0101. We can eliminate a static-0 hazard by adding a group (sum term) that covers the two 0's.

Hazards in a multi-level circuit can be detected by deriving either a SOP or POS expression for the circuit that represents a two-level circuit containing the same hazards as the original circuit. The SOP or POS expression is derived in the normal manner except that the complementation laws, i.e. $XX' = 0$ and $X + X' = 1$, are not used. Consequently:

- The resulting SOP expression may contain products of the form $xx'\alpha$, where α is a product of literals or nothing. In the SOP expression, a product of the form $xx'\alpha$ represents a pseudo AND gate that may temporarily have the output value 1 as x changes if $\alpha = 1$, which causes static-0 or dynamic hazards. A such hazard is denoted as the $xx'\alpha$ term.
- The resulting POS expression may contain sums of the form $x + x' + \beta$, where β is a sum of literals or nothing. In the POS expression, a sum of the form $x + x' + \beta$ represents a pseudo OR gate that may temporarily have the output value 0 as x changes if $\beta = 0$, which causes static-1 or dynamic hazards. A such hazard is denoted as the $x + x' + \beta$ term.

To design a circuit which is free of static and dynamic hazards, the following procedures may be used:

1. Find a SOP expression for the function in which every pair of adjacent 1's is covered by a 1-term. (The sum of all prime implicants always satisfies this condition.) A two-level AND-OR circuit based on this expression will be free of all hazards.
2. If a different form of the circuit is desired, manipulate the expression to the desired form but treat each independent variable and its inverse as two independent variables to prevent introduction of hazards.

or

1. Find a POS expression for the function in which every pair of adjacent 0's is covered by a 0-term. (The product of all prime implicate always satisfies this condition.) A two-level OR-AND circuit based on this expression will be free of all hazards.
2. If a different form of the circuit is desired, manipulate the expression to the desired form but treat each independent variable and its inverse as two independent variables to prevent introduction of hazards.

It should be emphasized that the discussion of hazards and the possibility of resulting glitches in this section has assumed that only a single input can change at a time and that no other input will change until the circuit has stabilized. If more than one input can change at one time, then nearly all circuits will contain hazards, and they cannot be eliminated by modifying the circuit implementation.

VII Simulation and Testing

i Verilog, SystemVerilog, and VHDL

A hardware description language (HDL) is a specialized computer language used to describe the structure and behavior of electronic circuits.

Verilog, standardized as IEEE 1364, is a hardware description language used to model and simulate electronic circuits.

SystemVerilog, an extension of Verilog, standardized as IEEE 1800, is a hardware description and hardware verification language used to model, simulate, and test electronic circuits.

VHDL (VHSIC Hardware Description Language), standardized as IEEE Std 1076, is a hardware description language used to model and simulate electronic circuits.

ii Four-valued logic

The two logic values, 0 and 1, are not sufficient for simulating logic circuits. At times, the value of a gate input or output may be unknown, which is represented by X (x in Verilog/SystemVerilog). At other times we may have no logic signal at an input, as in the case of an open circuit when an input is not connected to any output, called high impedance or hi-Z/Hi-Z connection, which is represented by Z (z in Verilog/SystemVerilog).

The logical connectives are defined as follows (without loss of commutativity of operators):

$$X \cdot 0 = Z \cdot 0 = 0,$$

$$X \cdot 1 = X \cdot X = X \cdot Z = Z \cdot 1 = Z \cdot Z = X,$$

$$X + 1 = Z + 1 = 1,$$

$$X + 0 = X + X = X + Z = Z + 0 = Z + Z = X.$$

iii Simulation and testing

A simple simulator for combinational logic works as follows:

1. The circuit inputs are applied to the first set of gates in the circuit, and the out-puts of those gates are calculated.
2. The outputs of the gates which changed in the previous step are fed into the next level of gate inputs. If the input to any gate has changed, then the output of that gate is calculated.
3. Step 2 is repeated until no more changes in gate inputs occur. The circuit is then in a steady-state condition, and the outputs may be read.
4. Steps 1 through 3 are repeated every time a circuit input changes.

If a circuit output is wrong for some set of input values, this may be due to several possible causes:

- Incorrect design
- Gates connected wrong
- Wrong input signals to the circuit

If the circuit is physically built, other possible causes include

- Defective gates
- Defective connecting wires

It is very easy to locate the problem systematically by starting at the output and working back through the circuit until the trouble is located.

VIII Integrated Circuit (IC)

i Integrated Circuit (IC), microcip, or chip

An integrated circuit (IC), also known as a microchip or simply chip, is a compact assembly of electronic circuits, in which the components are fabricated onto a thin piece (called chip) of semiconductor material, most commonly silicon.

Integrated circuits can be broadly classified into analog, digital and mixed-signal, consisting of analog and digital signaling on the same IC.

ii Integrated circuit package

An integrated circuit package is the physical case that holds the silicon chip (die) and provides the electrical and mechanical connection between the chip’ s microscopic circuits and the outside world. An IC package typically has:

- Silicon die (chip): The actual IC.
- Bond wires: Typically tiny gold or aluminum wires.

- Encapsulation: Protective material that shields the die from damage.
- Pins or leads: Metal terminal that connect the IC to a printed circuit board (PCB).

iii Scales

Acronym	Name	Year	Transistor count	L
SSI	small-scale integration	1964	1 to 10	
MSI	medium-scale integration	1968	10 to 500	
LSI	large-scale integration	1971	500 to 20000	
VLSI	very-large-scale integration	1980	20000 and more (or 20000 to 1000000)	10000 an
(ULSI	ultra-large-scale integration	1964	1000000 and more	

It is generally uneconomical to design digital systems using only SSI and MSI integrated circuits. By using LSI and VLSI ICs, the required number of integrated circuit packages is greatly reduced, and the cost of mounting, wiring, designing, and maintaining may be significantly lower.

IX Combinational circuits

i Multiplexer (MUX)

A multiplexer (MUX) or data selector has a group of data inputs and a group of select inputs (aka control inputs, select lines, or control lines) and one output. The select inputs are used to select one of the data inputs and connect it to the output terminal. Multiplexers are frequently used in digital system design to select the data which is to be processed or stored.

A 2^n-to-1 MUX has 2^n data inputs $D_{2^n-1}, D_{2^n-2}, \dots D_0$, n select inputs $S_{n-1}, S_{n-2}, \dots S_0$ with S_{n-1}, and one output Y, in which D_i is selected when the unsigned binary number $S_{n-1}S_{n-2}\dots S_0$ equal to i. (Some sources label the subscripts of the select inputs backwards, that is, S_0 being MSB and S_{n-1} being LSB.)

Y is given by

$$Y = \sum_{i=0}^{2^n-1} D_i \cdot m_i(S_{n-1}, S_{n-2}, \dots S_0),$$

where $m_i(S_{n-1}, S_{n-2}, \dots S_0)$ is the n-variable minterm of index i.

Equivalently,

$$Y = \prod_{i=0}^{2^n-1} \left(D_i + M_i(S_{n-1}, S_{n-2}, \dots S_0)\right),$$

where $M_i(S_{n-1}, S_{n-2}, \dots S_0)$ is the n-variable maxterm of index i.

The symbol of an 2^n-to-1 MUX in block diagrams is an isosceles trapezoid, sometimes a rectangle, with paper output assigned as positive z-axis, the ray from the longer parallel side

to the shorter parallel side perpendicular to them assigned as positive y axis, with the longer parallel side contains the data input pins ordered from D_0 to D_{2^n-1}, typically from negative x-axis to positive x axis, with the unsigned number i, typically in binary, labeled inside the trapezoid beside D_i, one of the legs, typically the leg farther from D_0, contains the select inputs ordered from MSB (S_{n-1}) to LSB (S_0) from closest to the longer parallel side to farthest from the longer parallel side, sometimes with name of each select input labeled inside the trapezoid beside each of them, and the shorter parallel side contains the output pin.

A multiplexer can be implemented with a two- or multi-level circuit that realizes the SOP or POS form given above. Alternatively, a 2^n-to-1 multiplexer can be realized with $\left(2^{n-m+1}-1\right)2^m$-to-1 multiplexers ($m \leq n$).

A MUX with active high outputs outputs the selected input directly, while a MUX with active low outputs outputs the inverse of the selected input.

A MUX can also have an additional enable input E and work as a normal MUX when $E = 1$ and always output 0 or 1 or hi-Z when $E = 0$ depending on the design.

Given a n-variable switching function F, 2^m m-variable subfunctions of F can be obtained using Shannon's expansion of the function and compose back to F with one 2^{n-m}-to-1 multiplexer. Take $m = 1$, we can get a realization of F with one 2^{n-1}-to-1 multiplexer.

ii Three-state buffer or tri-state buffer

A three-state buffer (or tri-state buffer) is a logic gate that has three stable states: logic HIGH (1), logic LOW (0), and high impedance (Z). In the high impedance state, the output of the buffer is effectively disconnected from the subsequenct circuit. A three-state buffer has one data input and one output like a simple buffer and one additional enable input signal En. It works as a simple buffer when $En = 1$ and outputs high impedance when $En = 0$.

When a bus is driven by three-state buffers, we call it a three-state bus.

We can also add an inversion bubble at enable signal such that it works as a simple buffer when $En = 0$ and outputs high impedance when $En = 1$. We can also add an inversion bubble at either data input or output such that when it doesn't outputs high impedance, it works as an inverter.

We can implement a 2-to-1 MUX by using data inputs of the MUX as data inputs of two tri-state buffers, using the select input and its inverse as enable signal of the two tri-state buffers respectively, and simply connecting the two outputs of the two tri-state buffers to the output of the MUX.

iii Bidirectional I/O pin

Integrated circuits are often designed using bidirectional (or bi-directional) I/O pins for input and output to save pins and wiring. A bidirectional pin is a single physical pin that can act as either an input or an output, but not both at the same time, depending on the control signal. To accomplish this, the circuit output is connected to the pin through a three-state buffer. When

the buffer is enabled, the pin is driven with the output signal; when the buffer is disabled, an external source can drive the pin.

iv Half adder (HA)

A half adder is a combinational circuit that has two inputs, A (augend bit) and B (addend bit), and produces two ouputs S (sum) and C_{out} (carry), in which

$$S = A \oplus B,$$

$$C_{out} = AB.$$

The symbol of a half adder in block diagrams is a square with ”half adder” in it with augend bit and addend bit inputs lines and sum output line on two parallel sides; let x-axis be the line perpendicular to the two sides oriented from augend bit and addend bit inputs lines to sum output line, z-axis be oriented out of the paper; carry output with arrows on one of the other side with direction of it in positivite y direction.

v Half subtractor (HS)

A half subtractor is a combinational circuit that has two inputs, A (minuend bit) and B (subtrahend bit), and produces two ouputs D (difference) and B_{out} (borrow), in which

$$D = A \oplus B,$$

$$B_{out} = A'B.$$

The symbol of a half subtractor in block diagrams is a square with ”half subtractor” in it with minuend bit and subtrahend bit inputs lines and difference output line on two parallel sides; let x-axis be the line perpendicular to the two sides oriented from minuend bit and subtrahend bit inputs lines to difference output line, z-axis be oriented out of the paper; borrow output with arrows on one of the other side with direction of it in positivite y direction.

vi Full adder (FA)

A full adder is a combinational circuit that has three inputs, A (augend bit), B (addend bit), and C_{in} (carry-in), and produces two ouputs S (sum) and C_{out} (carry-out), in which

$$S = A \oplus B \oplus C_{in},$$

$$C_{out} = AB + AC_{in} + BC_{in}.$$

It can be implemented with

$$X = A \oplus B.$$

$$S = X \oplus C_{in}.$$

$$C_{out} = AB + XC_{in}.$$

The symbol of a full adder in block diagrams is a square with ”full adder” in it with augend bit and addend bit inputs lines and sum output line on two parallel sides; let x-axis be the

line perpendicular to the two sides oriented from augend bit and addend bit inputs lines to sum output line, z-axis be oriented out of the paper; carry-in input and carry-out output with arrows on the other two sides respectively with direction of them in positivite y direction and from carry-in input to carry-out output.

A FA assumed carry-in being 0 is given by

$$S = A \oplus B,$$

$$C_{out} = AB.$$

A FA assumed carry-in being 1 is given by

$$S = A \odot B,$$

$$C_{out} = A + B.$$

A full adder maybe revised to have the same inputs as a full adder, some outputs removed, and some new outputs added. Some outputs commonly added are generate $G = AB$ and propagate $P = A \oplus B$.

vii Full subtractor (FS)

A full subtractor is a combinational circuit that has three inputs, A (minuend bit), B (subtrahend bit), and B_{in} (borrow-in), and produces two ouputs D (difference) and B_{out} (borrow-out), in which

$$D = A \oplus B \oplus B_{in},$$

$$B_{out} = A'B + A'B_{in} + BB_{in}.$$

It can be implemented with

$$X = A \oplus B.$$

$$D = X \oplus B_{in}.$$

$$B_{out} = A'B + X'B_{in}.$$

Proof.

$$B_{out} = A'B + X'B_{in} = A'B + ABB_{in} + A'B'B_{in} = A'B + BB_{in} + A'B_{in}.$$

□

The symbol of a full subtractor in block diagrams is a square with ”full subtractor” in it with minuend bit and subtrahend bit inputs lines and difference output line on two parallel sides; let x-axis be the line perpendicular to the two sides oriented from minuend bit and subtrahend bit inputs lines and difference output line, z-axis be oriented out of the paper; borrow-in input and borrow-out output with arrows on the other two sides respectively with direction of them in positivite y direction and from borrow-in input to borrow-out output.

viii Parallel (binary) adder, ripple-carry adder (RCA), or carry propagate adder (CPA)

An n-bit parallel adder or ripple-carry adder, or carry propagate adder is a combinational circuit consisting of n full adders called cells or stages. It has two n-bits unsigned or two's complement binary numbers as inputs (MSB to LSB), $A_{n-1}A_{n-2}\dots A_0$ (augend) and $B_{n-1}B_{n-2}\dots B_0$ (addned), and a constant 0 input, $C_{in_0} = 0$, and produces the $(n+1)$-bit sum of the two numbers as outputs, $C_{out_{n-1}}S_{n-1}S_{n-2}\dots S_0$, in which the $(i+1)$th cell takes A_i as augend bit, B_i as addend bit, and C_{in_i} as carry-in, and produces sum S_i and carry-out C_{out_i}, that is,

$$S_i = A_i \oplus B_i \oplus C_{in_i},$$

$$C_{out_i} = A_iB_i + A_iC_{in_i} + B_iC_{in_i},$$

where C_{in_i} for $n \geq i > 0$ and C_{out_i} for $n-1 > i \geq 0$ are interconnections in the parallel adder, and for the first cell,

$$S_0 = A_0 \oplus B_0,$$

$$C_{out_0} = A_0B_0.$$

The first cell can be replaced with a half adder.

The ripple-carry adder is relatively slow because, in the worst case, a carry propagates through all n stages of the adder before it stabilizes, and there are approximately two gate delays per stage.

A logical equivalent of an n-bit ripple-carry adder, that is, with two n-bits unsigned or two's complement binary numbers as inputs (MSB to LSB), $A_{n-1}A_{n-2}\dots A_0$ (augend) and $B_{n-1}B_{n-2}\dots B_0$ (addned), (and a C_{in_0} input set 0), and produces the $(n+1)$-bit sum of the two numbers as outputs, $C_nS_{n-1}S_{n-2}\dots S_0$, is called an n-bit adder.

ix Parallel (binary) subtractor, ripple-borrow subtractor (RBS), or borrow propagate subtractor (BPS)

An n-bit parallel subtractor, ripple-borrow subtractor, or borrow propagate subtractor is a combinational circuit consisting of n full subtractors called cells or stages. It has two n-bits unsigned or two's complement binary numbers as inputs (MSB to LSB), $A_{n-1}A_{n-2}\dots A_0$ (minuend) and $B_{n-1}B_{n-2}\dots B_0$ (subtrahend), and a constant 0 input, $B_{in_0} = 0$, and produces the $(n+1)$-bit two's complement difference of the two numbers as outputs, $B_{out_{n-1}}D_{n-1}D_{n-2}\dots D_0$, in which the $(i+1)$th cell takes A_i as minuend bit, B_i as subtrahend bit, and B_{in_i} as borrow-in, and produces difference D_i and borrow-out B_{out_i}, that is,

$$D_i = A_i \oplus B_i \oplus B_{in_i},$$

$$B_{out_i} = A_i'B_i + A_i'B_{in_i} + B_iB_{in_i},$$

where B_{in_i} for $n \geq i > 0$ and B_{out_i} for $n-1 > i \geq 0$ are interconnections in the parallel subtractor, and for the first cell,

$$D_0 = A_0 \oplus B_0,$$

$$D_{out_0} = A_0'B_0.$$

The first cell can be replaced with a half subtractor.

The ripple-borrow subtractor is relatively slow because, in the worst case, a borrow propagates through all n stages of the subtractor before it stabilizes, and there are approximately two gate delays per stage.

A logical equivalent of an n-bit ripple-borrow subtractor, that is, with two n-bits unsigned or two's complement binary numbers as inputs (MSB to LSB), $A_{n-1}A_{n-2}\dots A_0$ (minuend) and $B_{n-1}B_{n-2}\dots B_0$ (subtrahend), (and a B_{in_0} input set 0), and produces the $(n+1)$-bit two's complement difference of the two numbers as outputs, $B_{out_{n-1}}D_{n-1}D_{n-2}\dots D_0$, is called an n-bit subtractor.

x Adder-subtractor

A logical equivalent of a subtractor that is used more often is an adder with inversion bubbles at all B_i inputs and C_{in_0} being constant 1 input, since $B'_{n-1}B'_{n-2}\dots B'_0+1$ represents $-B_{n-1}B_{n-2}\dots B_0$ in two's complement.

An n-bit adder-subtractor is an n-bit adder with an additional common select input M and n 2-to-1 MUXes, in which the $(i+1)$th MUX takes data inputs B_i and B'_i and select input M, if $M=0$, it outputs B_i to to make the adder-subtractor perform addition; if $M=1$, it outputs B'_i to make the adder-subtractor perform subtraction.

xi One's complement adder

An n-bit one's complement adder can be realized with an n-bit ripple-carry adder but connect $C_{out_{n-1}}$ to C_{in_0} to realize end-around carry (EAC). The propagation delay when $C_{out_{n-1}}=1$ is approximately twice the propagation delay of the ripple-carry adder.

xii One's complement subtractor

An n-bit one's complement subtractor can be realized with an n-bit ripple-borrow subtractor but connect $B_{out_{n-1}}$ to B_{in_0} to realize end-around borrow (EAB). The propagation delay when $B_{out_{n-1}}=1$ is approximately twice the propagation delay of the ripple-borrow subtractor.

An n-bit one's complement subtractor can also be realized with a one's complement adder but invert all bits of the subtrahend.

xiii Carry-lookahead adder (CLA)

An n-bit hierarchical carry-lookahead adder is an n-bit adder designed to speed up addition to $O(\log n)$ by reducing the carry propagation delay.

Full adders for CLAs are revised to remove carry-out output and add generate $G=AB$ and propagate $P=A\oplus B$ outputs.

Suppose the maximum number of fan-ins of our add gates is m, we can implement carry-lookahead circuit up to $m-1$ bit. An $n<m$-bit carry-lookahead circuit has n generate inputs G_0

to G_{n-1}, n propagate inputs P_0 to P_{n-1}, one carry-in input C_0, and produces n carry-out outputs C_1 to C_n given recursively by

$$C_{i+1} = G_i + P_i C_i$$

but realized in expanded SOP form, which is $O(1)$ but limited by gate fan-ins. An $n < m$-bit carry-lookahead has n revised full adders called cells or stages and one n-bit carry-lookahead circuit, in which the $(i+1)$th cell takes A_i as augend bit, B_i as addend bit, and C_i as carry-in, and produces sum S_i, generate G_i, and propagate P_i, and the carry-lookahead circuit take generate inputs G_0 to G_{n-1}, propagate inputs P_0 to P_{n-1}, carry-in input C_0, and produces n carry-out outputs C_1 to C_n.

When the maximum number of fan-ins is not greater than the number of bit needed, we use hierarchical lookahead structure, in which the carry-lookahead circuit is replaced with a tree of physically possible m-bit carry-lookahead circuits. A m^k-bit carry-lookahead adder consists of k levels of carry-lookahead circuits, in which the lth level consists m^{k-l} m-bit carry-lookahead circuits, each of the carry-lookahead circuits that are not in the last level (which contains only one carry-lookahead circuit) is revised to output generate g given recursively by

$$g_0 = G_0,$$

$$g_{i+1} = G_{i+1} + P_i g_i,$$

$$g = g_{m-1},$$

and propagate p given by

$$p = \prod_{i=1}^{m-1} P_i,$$

instead of carry-out outputs, where m G_i and m P_i are the generates and propagates of the revised full adders ditributed in order to each of them, and each of the carry-lookahead circuits that are not in the first level takes m generates and m propagates of the previous level in order and computed the same as a normal m-bit carry-lookahead circuit by treating the generate and propagate of the $m \times (i-1) + j$th carry-lookahead circuit in the previous level as the jth generate and propagate inputs of the ith carry-lookahead circuit in this level.

xiv Carry-save adder (CSA)

A n-bit carry-save adder sums three n-bit operands $X_0, X_1, \ldots, X_{n-1}$, $Y_0, Y_1, \ldots, Y_{n-1}$, and $Z_0, Z_1, \ldots, Z_{n-1}$ with n parallel disconnected full adders by feeding the bits of each operand into the three inputs of a full adder (FA) to produce an n-bit sum vector $S_0, S_1, \ldots, S_{n-1}$ from sum outputs and an $(n+1)$-bit carry vector $0, C_1, C_2, \ldots, C_n$ (that is, shifted by one bit towards more significant bit with 0 padded at the LSB) from carry-out outputs, which is called a 3-to-2 reduction and is of $O(1)$ time complexity.

A multi-operand carry-save adder tree (or reduction tree) sums multiple operands and consists of layers of carry-save adders, called reduction layers, and a final adder, which is an adder, typically ripple adder or carry-lookahead adder, that sums two operands to produce the final result. For a carry-save adder tree that sums m n-bit operands with the k least significant bits of the result to be preserved ($n \le k \le n + \lceil \log_2 m \rceil$),

- the first reduction layer consists of $\left\lfloor \frac{m}{3} \right\rfloor$ n-bit carry-save adders, and produce $\left\lfloor \frac{m}{3} \right\rfloor$ n-bit sum vectors and $\left\lfloor \frac{m}{3} \right\rfloor$ $(n+1)$-bit carry vectors, where if the carry vectors exceed k bits, the bits that are more significant than the kth least significant bit are discarded,
- the remaining reduction layers form a carry-save adder tree that sums $2\left\lfloor \frac{m}{3} \right\rfloor + m \mod 3$ $\min((n+1), k)$-bit operands with the k least significant bits of the result to be preserved, and
- the final adder is k-bit and sums the sum vector and carry vector from the last reduction layer to produce the final result.

Let the time complexity of the final adder be $O(f(k))$. The time complexity of the CSA method is $O(\log m + f(k))$.

xv Carry-select adder (CSLA)

An n-bit carry-select adder consists of carry-select blocks indexed from least significant block to most significant block from 1 to b such that with the number of bit of the block of index i be n_i, it satisfies

$$\sum_{i=1}^{b} n_i = n.$$

An m-bit block of index $i > 1$ consists of two m-bit ripple carry adders where the first with C_{in_0} assumed 0 and the second with C_{in_0} assumed 1, and $(m+1)$ 2-to-1 MUXes that, for each bit, with one MUX taking the sum outputs of the two ripple carry adders as data input to produce the final sum output of the bit, and with one MUX taking the C_{out_m} of the two ripple carry adders as data input to produce the carry-out output of the block (or, for the block of index b, the $(n+1)$th bit of the final sum), where all MUXes take the carry-out output from the block of index $(i-1)$. An m-bit least significant block is an m-bit ripple carry adder that produces real sum outputs and carry-out output of the block.

Assume the propagation delay of the ripple carry adder in the block of any index $i > 1$ is not greater than the propagation delay of the ripple carry adder in the block of index 1 plus $(i-1)$ times the propagation delay of a 2-to-1 MUX. Then the total propagation delay is the propagation delay of the ripple carry adder in the block of index 1 plus $(b-1)$ times the propagation delay of a 2-to-1 MUX.

For carry-select adder with fixed block size, say k bit, and thus $b = \frac{n}{k}$, the time complexity is $O\left(k + \frac{n}{k}\right)$, and by

$$\frac{\mathrm{d}}{\mathrm{d}k}\left(k + \frac{n}{k}\right) = 1 - \frac{n}{k^2},$$
$$k = \sqrt{n},$$
$$k + \frac{n}{k} = 2\sqrt{n},$$

the optimal time complexity is $O(\sqrt{n})$ and the optimal such carry-select adder is of block size $\sqrt{n}$ bits.

The speed can be further increased by making middle blocks larger than end blocks.

xvi Carry-skip adder (CSkA) or carry-bypass adder

An n-bit carry-skip adder consists of carry-skip blocks indexed from least significant block to most significant block from 1 to b such that with the number of bit of the block of index i be n_i, it satisfies

$$\sum_{i=1}^{b} n_i = n.$$

A m-bit block of index i is an m-bit ripple carry adder with full adders revised to add propagate $P = A \oplus B$ output, a block propagate P_{bi} that is the propagate outputs of all revised full adders in the block ANDed together, and a MUX that outputs the carry-out output of this block with P_{bi} as the select input that selects the carry-out output $P_{b(i-1)}$ from block of index $(i-1)$ (or 0 for $i = 1$) when $P_{bi} = 1$, called bypass or skip, and selects the carry-out output of the most significant revised full adder of the ripple carry adder of this block when $P_{bi} = 0$.

For carry-skip adder with fixed block size, say s bit, and thus $b = \frac{n}{s}$, the worst-case total propagation delay is two times the propagation delay of the ripple carry adder in a block plus the propagation delay of $P_{b(b-1)}$, that is, $O\left(2s + \frac{n}{s}\right)$, and by

$$\frac{\mathrm{d}}{\mathrm{d}s}\left(2s + \frac{n}{s}\right) = 2 - \frac{n}{s^2},$$

$$s = \sqrt{\frac{n}{2}},$$

$$2s + \frac{n}{s} = 2\sqrt{2n},$$

the optimal worst-case time complexity is $O(\sqrt{n})$, and such adder is of block size $\sqrt{\frac{n}{2}}$ bits.

The speed can be further increased by making middle blocks larger than end blocks.

xvii Parallel prefix adder (PPA)

Parallel prefix adders are a class of adders consist of three stages, preprocessing stage, prefix network or prefix computation stage, and postprocessing stage. For an n-bit parallel prefix adder adding operands $A_0A_1 \ldots B_{n-1}$ and $B_0B_1 \ldots B_{n-1}$, preprocessing stage computes generate $G_i = A_i \cdot B_i$ and propagate $P_i = A_i \oplus B_i$ for each bit, and then a prefix operation $\circ$ is defined as

$$(G_k, P_k) \circ (G_j, P_j) = (G_k + P_k G_j, P_k P_j),$$

which is associative, and the prefix network starts with n roots, (G_0, P_0) to (G_{n-1}, P_{n-1}), to compute n goals,

$$(G_i^*, P_i^*) \coloneqq \operatorname*{bigcirc}_{j=0}^{i}\left(G_{i-j}, P_{i-j}\right)$$

for all $i \in \mathbb{Z} \cap [0, n-1]$, where $C_{i+1} = G_i^* + P_i^* C_0$, each route from (G_i, P_i) to (G_i^*, P_i^*) for all $i \in \mathbb{Z} \cap [0, n-1]$ is hereafter called the i span, and the data in the i span is hereafter denoted (g_i, p_i), and finally, the postprocessing stage computes $S_i = P_i \oplus C_i$ for all $i \in \mathbb{Z} \cap [0, n-1]$. For $i \neq j$, that the j span is combined with i span means the j span is replaced with $(g_j, p_j) \circ (g_i, p_i)$ and the i span is unchanged. This class of adder has time complexity $O(\log n)$, which is optimal. Different

designs differ in the prefix network stage and can be different in area, wiring complexity, and fan-out.

xviii Kogge–Stone adder (KSA or KS)

Kogge–Stone adder is a type of parallel prefix adder optimized for minimum logic depth but with larger area and wiring complexity. The prefix network of it consists of $\lceil \log_2(n) \rceil$ stages. For all $i \in [1, \lceil \log_2(n) \rceil - 1]$, in the ith stage, for $j \in \mathbb{Z} \cap [2^{i-1}, n-1]$, the j span is combined with the $j - 2^{i-1}$ span. The fan-out is $\lceil \log_2(n) \rceil + 1$.

xix Brent–Kung adder (BKA or BK)

Brent–Kung adder is a type of parallel prefix adder with smaller area and wiring complexity but longer logic depth than Kogge–Stone adder. The prefix network of it is split into reduction or up-sweep phase, with $\lceil \log_2(n) \rceil$ stages, and distribution or down-sweep phase, with $\lceil \log_2(n) \rceil - 1$ stages, which are $2\lceil \log_2(n) \rceil - 1$ stages in total. Suppose $n = 2^k$. For all $i \in \mathbb{Z} \cap [1, k]$, in the ith stage of reduction phase, for all $j \in \mathbb{Z} \cap [1, 2^{k-i}]$, the $j * 2^i - 1$ span is combined with the $j * 2^i - 2^{i-1} - 1$ span. For all $i \in \mathbb{Z} \cap [1, k-1]$, in the ith stage of distribution phase, for all $j \in \mathbb{Z} \cap [1, 2^i - 1]$, the $j * 2^{k-i} + 2^{k-i-1} - 1$ span is combined with $j * 2^{k-i} - 1$ span. The fan-out is $\lceil \log_2(n) \rceil$.

xx Han–Carlson adder (HCA)

Han–Carlson adder is a type of parallel prefix adder that combines the ideas of Kogge–Stone and Brent–Kung adder. The prefix network of it is split into three phases, Brent–Kung reduction or up-sweep phase, Kogge–Stone phase, and Brent–Kung distribution or down-sweep phase. Suppose $n = 2^k$. A particular family of one parameter $s \in \mathbb{Z} \cap [1, k)$ of Han–Carlson adder is described as follows. Both Brent–Kung reduction phase and Brent–Kung distribution phase have s phases, and Kogge–Stone phase has $k - s$ phases. For all $i \in \mathbb{Z} \cap [1, s]$, in the ith stage of Brent–Kung reduction phase, for all $j \in \mathbb{Z} \cap [1, 2^{k-i}]$, the $j * 2^i - 1$ span is combined with the $j * 2^i - 2^{i-1} - 1$ span. For all $i \in [1, k-s]$, in the ith stage of Kogge–Stone phase, for $j \in \mathbb{Z} \cap [2^{i-1} + 1, 2^{k-s}]$, the $j * 2^s - 1$ span is combined with the $(j - 2^{i-1}) * 2^s - 1$ span. For all $i \in \mathbb{Z} \cap [1, s]$, in the ith stage of Brent–Kung distribution phase, for all $j \in \mathbb{Z} \cap [1, 2^{k-s-1+i} - 1]$, the $j * 2^{s+1-i} + 2^{s-i} - 1$ span is combined with $j * 2^{s+1-i} - 1$ span. The number of stages in total is $k + s$. The fan-out is $k + 1$.

xxi Sklansky adder (SA) or divide-or-conquer adder

Sklansky adder is a type of parallel prefix adder optimized for minimum logic depth and with smaller area and wiring complexity than Kogge–Stone adder but has very high fan-out of $n - 2^{\lceil \log_2(n) \rceil - 1} + 1$. The prefix network of it consists of $\lceil \log_2(n) \rceil$ stages. For all $i \in \mathbb{Z} \cap [1, 2^{\lceil \log_2(n) \rceil - i}]$, for all $l \in [j * 2^i - 2^{i-1}, \min(j * 2^i - 1, n-1)]$, the l span is combined with the $j * 2^i - 2^{i-1} - 1$ span, where the $j * 2^i - 2^{i-1} - 1$ span has fan-out of $\min\left(2^{i-1} + 1, n - 2^{\lceil \log_2(n) \rceil - 1} + 1\right)$.

xxii Decoders

A decoder is a combinational circuit that converts binary input codes into a single active output line. An n-to-2^n decoder has n input signals and 2^n output signals realizing a one-to-one Boolean vector function $\{0,1\}^n \to \{0,1\}^{2^n}$ that map each input combination to an output vector of which the 1-norm is 1 (noninverted/active-high outputs) or $2^n - 1$ (inverted/active-low outputs).

Each possible output of a decoder corresponds to a minterm of the inputs. Thus, we can OR them together to get the active-high outputs or NAND them together to get the active-low outputs.

The symbol of an n-to-2^n decoder in block diagrams is a rectangle with inputs lines and output lines with arrows on two parallel sides and "(n-to-2^n) decoder" in it.

xxiii Normal encoders

A normal encoder is a combinational circuit that realize the inverse of a Boolean vector function realized by a decoder. A 2^n-to-n normal encoder realizes the inverse of a Boolean vector function $\{0,1\}^n \to \{0,1\}^{2^n}$ realized by a decoder, of which the domain of the inverse is a subset of $\{0,1\}^{2^n}$ that has n elements, each of which with the 1-norm being 1 (noninverted/active-high inputs) or $(2^n - 1)$ (inverted/active-low inputs). If the input combination is not in the domain, the outputs are undefined.

The symbol of a 2^n-to-n normal encoder in block diagrams is a rectangle with inputs lines and output lines with arrows on two parallel sides and "(2^n-to-n) normal encoder" in it.

xxiv Priority encoders

A priority encoder resolves the undefined behaviour of normal encoders by assigning priority to the inputs.

A 2^n-to-n active-high inputs and outputs priority encoder has 2^n inputs and $(n+1)$ outputs. If all inputs are 0, the $(n+1)$-th output, called valid (V) or enable output (EO), is 0 and the other n outputs are undefined; otherwise the $(n+1)$-th output is 1 and the other n outputs are determined by a loop: `for (int i=1; i<=2n; i=i+1)` (in which 2^n means 2 to the power of n):

- If the input of ith priority is 1, the output combination is the output combination of a active-high inputs normal encoder when that input is 1 and all other inputs are 0.

The priority of inputs are usually as: for any $1 \leq i < 2^n$, the $(i+1)$-th input is prior to the i-th input.

An active-low inputs priority encoder invert the inputs; an active-low outputs priority encoder invert the outputs.

The symbol of a 2^n-to-n priority encoder in block diagrams is a rectangle with inputs lines and output lines with arrows on two parallel sides and "(2^n-to-n) priority encoder" in it.

xxv Comparator

An n-bit comparator has two n-bit parallel input $A_{n-1}A_{n-2}\dots A_0$, $B_{n-1}B_{n-2}\dots B_0$ representing two n-bit unsigned binary numbers A and B and three output signals gt, which is 1 iff A>B, eq, which is 1 iff A=B, and lt, which is 1 iff A<B. It consist of n cells interconnected linearly. Each cell i has four inputs, gt_{i+1}, lt_{i+1}, A_i, and B_i, and two outputs, gt_i and lt_i, where

$$gt_i = gt_{i+1} + A_i B_i' lt_{i+1}',$$

$$lt_i = lt_{i+1} + A_i' B_i gt_{i+1}'.$$

However, the left end cell (cell $n-1$) has no input gt_n and lt_n, and

$$gt_{n-1} = A_i B_{n-1}',$$

$$lt_{n-1} = A_i' B_{n-1}.$$

Comparison proceeds from left to right, $gt = gt_0$, $lt = lt_0$, and $eq = gt \odot lt$.

xxvi Lookup table (LUT)

A lookup table is a possibly multidimensional array implemented in software or hardware that replaces runtime computation of a function with an array indexing retrieval, where the dimension of the array is the arity of the function.

X Devices

i Read-only memory (ROM)

A read-only memory (ROM) is a non-volatile (data is kept even when power is off) memory consists of an array of semiconductor devices that are interconnected to store an array of binary data. Once binary data is stored in the ROM, it can be read out whenever desired, but the data that is stored cannot be changed under normal operating conditions.

A k(-word)×m-bit ROM has $n = \lceil \log_2 k \rceil$ input lines and m output lines, and contains an array of k words, each word of width m bits, where k often equals 2^n. An input combination serves as an address to select one of the k words and output it from the ROM, that is, m n-variable functions are stored in it.. Typical sizes for commercially available ROMs range from 32 words × 4 bits to 512K words × 8 bits, or larger.

A k-word×m-bit ROM stores that has $n = \lceil \log_2 k \rceil$ input lines consist of an n-to-2^n decoder and a $k \times m$ memory array, aka OR plane. The latter consists of k rows, called word lines and each of which connected to some of the decoder output lines, and m columns, which are output lines of the ROM and connect to ground at the opposite end. In each of the $k \times m$ row-column intersection points, which corresponds to one bit, a diode, as a switching element, is either present and connected or not. If absent or not connected (logic 0), the signal value from the row isn't passed to the column; if present and connected (logic 1), represented by a × on the diagram, the signal value from the row is passed to the column. An output line outputs 1 if there are logic 1 signals from the word lines passed to it and 0 if there isn't.

The ROM table of a ROM is the truth tables of the functions realized combined by combining rows with same input combinations and listing the outputs of the functions in columns.

The symbol of a k-word×m-bit ROM in block diagrams is a rectangle with inputs lines and output lines with arrows on two adjacent sides and "(k-word×m-bit) ROM" in it, or an n-to-2^n decoder and a $k \times m$ memory array, with the latter being a rectangle with word lines with arrows, which are connected to the decoder outputs, and output lines with arrows, which are connected to ground at the starting end, on two adjacent sides.

Common types of ROMs:

- Mask-programmable ROM: Programmed permanently during manufacturing using photolithography by selectively including or omitting the diodes at the cells. The mask used in photolithography is expensive, so the use of mask-programmable ROMs is economically feasible only if a large quantity (typically several thousand or more) is required with the same data array.
- Programmable ROM (PROM): Manufactured with diodes at all cells and programmable once (one-time programmable (OTP)) by the user using a PROM programmer (aka PROM burner), which sends high-voltage pulses to burn tiny fuses in the cell to create permanent 1's and 0's.
- Erasable Programmable ROM (EPROM): Manufactured with each cell contains a control gate that receives normal programming or read voltages and a floating gate that is completely insulated by oxide and can store electric charge and with quartz window on top, programmable by the user using a PROM programmer (aka PROM burner), which sends high-voltage pulses to inject electrons into floating gates, and erasable by the user by expose it to intense UV light that provides photons providing energy to remove trapped electrons from the floating gate simultaneously at all celles.
- Electrically Erasable Programmable ROM (EEPROM): Similar to EPROM but erasing process is done electrically to remove trapped electrons from the floating gate one bit or one byte at a time (instead of all cells simultaneously) only a limited number of times, typically 100 to 1000 times.

ii Programmable Logic Devices (PLDs)

A programmable logic device (PLD) is a general name for a digital integrated circuit capable of being programmed to provide a variety of different logic function. Simple combinational PLDs are capable of realizing from 2 to 10 functions of 4 to 16 variables with a single integrated circuit. More complex sequential PLDs may contain thousands of gates and flip-flops. When a digital system is designed using a PLD, changes in the design can easily be made by changing the programming of the PLD without having to change the wiring in the system, which leads to lower cost designs.

iii Programmable Logic Arrays (PLAs)

A programmable logic array (PLA) performs the same basic function as a ROM. However, the internal organization of the PLA is different from that of the ROM. The decoder is replaced with an AND array which realizes selected product terms of the input variables. The OR array ORs together the product terms needed to form the output functions, so a PLA implements a sum-of-products expression, while a ROM directly implements a truth table. Product terms are formed in the AND array by connecting diodes as switching elements at appropriate points in the array. Outputs are formed in the OR array by connecting diodes as switching elements at appropriate points in the array.

A k(-input)×m(-word(-by))×n(-output) PLA has k inputs, m product terms (aka words or minterms), and n outputs, and can realize n k-variable functions in POS form with at most m product terms in total, where product terms in different functions may be shared. A product term with i literals needs i diodes in the AND array. A sum form with j product terms needs j diodes in the OR array.

The contents of a PLA can be specified by a PLA table that lists all product terms in it. A PLA table of an n-output PLA consists of three big columns. The first big column are product terms written in product of literals. The second big columns are input combinations of the product terms with $-$ indicating don't care. The third big column consists of n small columns, each of which contains one output of the PLA, where if the product term is connected by a diode to that output line, enter 1; otherwise, enther 0. Sometimes the first big column is omitted.

The symbol of a $k \times m \times n$ PLA in block diagrams is k input lines corresponding to each variable, each with variable name labeled at one end, indicating input side, each then split into two lines with one of them passing an inverter and the other not, forming AND array, n output lines parallel to the $2k$ AND array input lines, each with output name labeled at the end other than those of the input lines, indicating output side, forming OR array, and m word lines perpendicular to the $2k$ AND array input lines and the n output lines passing through them forming $2k \times m$ AND-array intersections, with a $\times$ at the intersection if a diode is connected there in the AND array, and $n \times m$ OR-array intersections, with a $\times$ at the intersection if a diode is connected there in the OR array.

When the number of input variables is small, a PROM may be more economical to use than a PLA. However, when the number of input variables is large, PLAs often provide a more economical solution than PROMs.

Type:

- Mask-programmable logic arrays: Programmed permanently during manufacturing using photolithography. The mask used in photolithography is expensive, so the use of mask-programmable logic arrays is economically feasible only if a large quantity (typically several thousand or more) is required with the same AND and OR arrays.
- Field-programmable logic arrays (FPLAs): Manufactured blank and programmable by the user. Technology of EPROM, EEPROM, or Flash memory may be used. Typically slower

and larger than mask-programmable logic arrays.

iv Programmable Array Logic (PAL)

A programmable array logic (PAL) is similar to a programmable logic array but in which the AND array is programmable and the OR array is fixed. PALs are less expensive than the more general PLAs and easier to program. For this reason, logic designers frequently use PALs to replace individual logic gates when several logic functions must be realized.

A buffer is used because each PAL input must drive many AND gate inputs. When the PAL is programmed, some of the interconnection points are programmed to make the desired connections to the AND gate inputs, represented by X's on the diagram.

When designing with PALs, we must simplify our logic equations and try to fit them into one (or more) of the available PALs. Unlike the more general PLA, the AND terms cannot be shared among two or more OR gates; therefore, each function to be realized can be simplified by itself without regard to common terms. For a given type of PAL, the number of AND terms that feed each output OR gate is fixed and limited. If the number of AND terms in a simplified function is too large, we may be forced to choose a PAL with more gate inputs and fewer outputs.

v Complex Programmable Logic Devices (CPLDs)

As integrated circuit technology continues to improve, more and more gates can be placed on a single chip. This has allowed the development of complex programmable logic devices (CPLDs). Instead of a single PAL or PLA on a chip, many PALs or PLAs can be placed on a single CPLD chip and interconnected. When storage elements such as flip-flops are also included on the same IC, a small digital system can be implemented with a single CPLD.

Take Xilinx XCR3064XL CPLD for example. This CPLD has four function blocks, each of which has 16 associated macrocells and is a programmable AND-OR array that is configured as a PLA. Each macrocell contains a flip-flop and multiplexers that route signals from the function block to the input-output (I/O) block or to the interconnect array (IA). The IA selects signals from the macrocell outputs or I/O blocks and connects them to function block inputs. The I/O blocks connect the signals from the interior of the CPLD to the bi-directional I/O pins on the IC.

vi Field-Programmable Gate Arrays (FGPAs)

An FPGA is an IC that consists of an array of identical logic cells, also called configurable logic blocks (CLBs) or function generators, with programmable interconnections and surrounded by I/O blocks that connect the CLB signals to IC pins. The user can program the functions realized by each logic cell and the connections between the cells. Each CLB contains a lookup table (LUT), flip-flops, and multiplexers. An n-input LUT is essentially a reprogrammable 2^n-word-$\times$-1-bit ROM that stores the truth table for the n-variable function being generated.

Given a n-variable switching function F and a FGPA with m-input LUTs ($m < n$), 2^{n-m} m-variable subfunctions of F can be obtained using Shannon's expansion of the function and compose

back to F with one 2^{n-m}-to-1 multiplexer, which can be composed of $\left(2^{n-m-k+1}-1\right)\ 2^k$-to-1 multiplexers ($k \leq n-m$).

3 Sequential Circuit

I Introduction

i Clock (CLK) signal

A clock signal is a pulse wave that oscillates between a high state, representing a ”one”, and a low state, representing a ”zero”, at a constant frequency. The period is called clock period or clock cycle. The ratio of high period to total period is called duty cycle.

Clock signals are generated by clock generators, which can be crystal oscillator (very stable, used in CPUs), RC oscillator (simpler and less precise), external clock generator, etc., and can be modified by other components in frequency or phase.

ii Synchronous and asynchronous circuit

A synchronous circuit or a clocked sequential is a sequential circuit where all state changes are coordinated by a single, global clock signal.

An asynchronous circuit is a sequential system that is not a synchronous circuit.

iii Latch

A latch is a bistable sequential circuit that store a single bit of information and hold its value until it is updated by new input signals. One of its two states represents a logic 0 and the other represents a logic 1.

For a latch to capture the inputted information and store it is called latch.

iv Asynchronous, Edge-triggered, and Level-triggered latch

An asynchronous latch (aka a basic latch) is a latch that responds to changes of data inputs all the time.

A level-triggered latch/flip-flop is a latch that responds to changes of data inputs when a signal, called enable (En or E) or clock (Clk, Ck, or C), is at a specific high or low level.

A edge-triggered flip-flop is a latch that responds to changes of data inputs only at the rising or falling edge of a clock signal, called clock (Clk, Ck, or C).

v Gated or level-sensitive latch

A gated or level-sensitive latch is a level-triggered latch that has an additional enable input signal En which controls whether the latch is transparent (enabled) or hold (disabled). When enable is at active level (1 for active-high gated latch and 0 for active-low gated latch), it is transparent, meaning the it acts as an asynchronous latch of its type; when enable is at inactive

level (0 for active-high latch and 1 for active-low latch), the latch is hold (aka retain), meaning the output remains at its previous state regardless of input changes.

Unless otherwise specified, a gated latch is active-high. A active-low gated latch can be implemented by adding an inverter at the enable input. The symbol of an active-low latch in block diagrams is that of an active-high one with an inversion bubble at the enable input right outside of the rectangle.

vi (Edge-triggered) Flip-flop (FF)

The term flip-flop has historically referred generically to both level-triggered and edge-triggered latch. Some modern authors, however, reserve the term flip-flop exclusively for edge-triggered latch and use the term gated latch or simply latch for level-triggered latch. The terms ”edge-triggered”, and ”level-triggered” may be used to avoid ambiguity.

The implementation a type of flip-flop with another type of flip-flop is called conversion.

vii Metastable state

A metastable state is a state where the state variables in a sequential circuit

- are constants and can theoretically stay as is if no noise occur, and
- transit away to a truly stable state if any noise occurs,

where a truly stable state is a state where the state variables return to the value of them in the state if any noise occurs.

The situation for a latch to be in a metastable state is called metastability.

The probability that metastability lasts decays exponentially over time.

Take the sequential circuit consists of two inverters with the output of one being the input of another for example. There are two stable states (excluding metastable state), 0 and 1, and one metastable state, a voltage right in the middle of 0 and 1.

viii Latch response time or delay time

The response time or delay time ϵ of an asynchronous or level-triggered latch is the time it needs for its state to react to a input change. If an impulse of input lasts less than ϵ, the latch or flip-flop will not react to it.

ix Flip-flop propagation delay, setup time, and hold time

The propagation delay t_p of a flip-flops is the time it takes for a change at the inputs to propagate to the output(s).

The setup time t_s of a flip-flop is the minimum time before an acitve edge that the data inputs must be stable so that the flip-flop can correctly capture it.

The hold time t_h of a flip-flop is the minimum time afte an acitve edge that the data inputs must be stable to ensure the flip-flop captures it correctly.

Violating setup or hold times can cause metastability.

A synchronizer can be used to synchronize an input to a synchronous circuit with its clock.

x State variable

A state variable is a binary variable that represents the current state of a sequential circuit.

The state variable stored in a latch is usually denoted as Q and sometimes called the same as the latch.

xi Present state and next state

When discussing latches and flip-flops, we the term present state $Q = Q(t)$ to denote the state of the Q output of the latch or flip-flop at the time input signal changes, and the term next state $Q^+ = Q(t + \epsilon)$ to denote the state of the Q output after the latch or flip-flop has reacted to the input change and stabilized.

xii Next-state Equation or Characteristic Equation

The equation of the next state of a sequential circuit as a function of the present state and inputs.

xiii Essential hazard

An essential hazard in an asynchronous circuit is a hazard, which causes glitch but does not affect the final stable state, caused by unequal propagation delays along at least two different paths that originate from the same input, at least one of which in the circuit's feedback paths. These hazards cannot be corrected by adding redundant logic gates and require adjusting the delays in the affected paths to be resolved.

xiv Race condition, race hazard, or race

A race condition, race hazard, or simply race is the condition which the behavior of a system is dependent on the sequence or timing of other uncontrollable events.

A non-critical race is a race which the final stable state is not dependent on the sequence or timing of other uncontrollable events.

A critical race is a race which the final stable state is dependent on the sequence or timing of other uncontrollable events. A critical race is caused by race of state variables and cannot be removed by adding redundant gates or adjusting the delays. To eliminate critical races, we can assign states in a way such that at most one state variable changes at a time, e.g., gray code.

xv Cross-Coupled Gates

To get the truth table of a sequential circuit with cross-coupled gates,

1. If the circuit composed of sequential subcircuits and combination subcircuits, we may analyze each sequential subcircuit respectively. It is usually easier to analyze a latch.

2. Solve the system of Boolean equations of input signals and output signals given by the gates.
3. For each combination of input signals,
    - If there is no root with them, the combination cause the circuit to be unstable, and signal tracing may be required to determine the behavior.
    - If there is only one root with them, all present states stabilize to the root.
    - If there are multiple roots with them, for each present state, signal tracing may be required to determine which root it will stabilize to. For the cases where the present combination of output signals exists in one of the roots, the original output signals are preserved.

Note that even if it eventually stabilizes to a state, glitches may happen beforehand, and signal tracing may be required to determine the glitches.

xvi Logic-equivalent circuit using standard latches and flip-flops

If only stable states are cared (i.e., glitches and behaviors when not stabilized are not cared), all sequential circuits have logic-equivalent circuits purely consist of standard latches and flip-flops (S-R, D, J-K, and T) and combinational subcircuits. We can thus analyze and design a such circuit and convert to the wanted circuit afterwards.

II Latches

i (Active-high) S-R Latch

An (asynchronous) (active-high or NOR-gate) set-reset (aka S-R or SR) latch or an (asynchronous) S-R NOR latch has two inputs, set S and reset R, and two outputs, Q and Q', and stores one bit, Q. It consists two NOR gates with S and Q being the inputs and Q' being the output of one NOR gate, and R and Q' being the inputs and Q begin the output of the other NOR gate.

It has a restriction that $SR = 0$.

With present state Q, next state Q^+, and next state of Q' P, the next-state equation of an S-R latch is:

$$Q^+ = (R + (S + Q)')' = R'S + R'Q.$$

$$P = (S + Q^+)' = S'Q^{+\prime}.$$

$$Q^{+\prime} = (R'S + R'Q)' = (R'S)'(R'Q)' = (R + S')(R + Q') = R + S'Q'.$$

$$P = S'R + S'Q'.$$

The truth table is:

S	R	Q	Q^+	$Q^{+\prime}$	P
0	0	0	0	1	1

0	0	1	1	0	0
0	1	0	0	1	1
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	1	0	0
1	1	0	0	1	0
1	1	1	0	1	0

Thus, $P = Q^{+\prime}$ except when $S = R = 1$.

By making $(S, R) = (1, 1)$ don't care combination, we can simplify them as:

$$Q^+ = S + R'Q.$$

$$Q^{+\prime} = P = R + S'Q'.$$

When $(S, R) = (1, 0)$, it sets Q^+ to 1; when $(S, R) = (0, 1)$, it resets Q^+ to 0; when $(S, R) = (0, 0)$, it holds present state; when $(S, R) = (1, 1)$, which is not allowed, the outputs oscillate.

The symbol of an active-high S-R latch in block diagrams is an rectangle with two input lines with label S and R in the rectangle adjacent to it on one long side, output line with label Q in the rectangle adjacent to it at the position opposite to S on the opposite side, and output line with label Q' in the rectangle adjacent to it at the position opposite to R on the opposite side.

ii Active-low S-R Latch

An (asynchronous) active-low or NAND-gate S-R latch, an (asynchronous) S-R NAND latch, $\overline{\text{SR}}$ latch, or $\overline{\text{S}}$-$\overline{\text{R}}$ latch has two inputs, set $\overline{S}$ and reset $\overline{R}$, and two outputs, Q and Q', and stores one bit, Q. It consists two NAND gates with $\overline{S}$ and Q' being the inputs and Q being the output of one NAND gate, and $\overline{R}$ and Q being the inputs and Q' begin the output of the other NAND gate.

It has a restriction that $\overline{S} + \overline{R} = 1$.

With present state Q, next state Q^+, and next state of Q' P, the next-state equation of an S-R latch is:

$$Q^+ = (\overline{S}(\overline{R}Q)')' = \overline{S}' + \overline{R}Q.$$

$$P = (\overline{R}Q^+)' = \overline{R}' + Q^{+\prime}.$$

$$Q^{+\prime} = (\overline{S}' + \overline{R}Q)' = \overline{S}(\overline{R}Q)' = \overline{S}\overline{R}' + \overline{S}Q'.$$

$$P = \overline{R}' + \overline{S}Q'.$$

The truth table is:

$\overline{S}$	$\overline{R}$	Q	Q^+	$Q^{+\prime}$	P
0	0	0	1	0	1
0	0	1	1	0	1
0	1	0	1	0	0
0	1	1	1	0	0
1	0	0	0	1	1
1	0	1	0	1	1
1	1	0	0	1	1
1	1	1	1	0	0

Thus, $P = Q^{+\prime}$ except when $\overline{S} = \overline{R} = 0$.

By making $(\overline{S}, \overline{R}) = (0, 0)$ don't care combination, we can simplify it as:

$$Q^{+\prime} = P = \overline{R}' + \overline{S}Q'.$$

When $(\overline{S}, \overline{R}) = (0, 1)$, it sets Q^+ to 1; when $(\overline{S}, \overline{R}) = (1, 0)$, it resets Q^+ to 0; when $(\overline{S}, \overline{R}) = (1, 1)$, it holds present state; when $(\overline{S}, \overline{R}) = (0, 0)$, which is not allowed, the outputs oscillate.

The symbol of an active-low S-R latch in block diagrams is an rectangle with two input lines with label $\overline{S}$ and $\overline{R}$ in the rectangle adjacent to it on one long side, output line with label Q in the rectangle adjacent to it at the position opposite to $\overline{S}$ on the opposite side, and output line with label Q' in the rectangle adjacent to it at the position opposite to $\overline{R}$ on the opposite side.

iii Switch Debouncing with an S-R Latch

When you press or release a physical pushbutton or toggle switch, the conductor contacts don't make or break cleanly but bounce (open-close-open-close rapidly), called switch bounce or contact chatter, which may cause noise transitions in logic instead of one if fed directly into digital system.

To ensure that each physical press or release of a switch produces only one transition in logic, we can connect logic 1 (+V) to a double throw switch, connect the two outputs b and a of the double throw switch to S and R of an S-R latch respectively, connect S and R to pull-down resistors, and connect Q to final output. When switch is switched from a to b, the following sessions happen on the time diagram:

1. switch at a: $S = 0$, $R = 1$, $Q = 0$,
2. bounce at a: $S = 0$, R bounces, $Q = 0$,
3. switch between a and b: $S = R = Q = 0$,

4. bounce at b: S bounces, $R = 0$, $Q = 1$ after delay time of the latch since bounce at b starts (delay time of the latch is longer than the time S bounces from one state to another),
5. switch at b: $S = 1$, $R = 0$, $Q = 1$.

iv Gated S-R Latch

A gated S-R latch has three inputs, set S, reset R, and enable E, and two outputs, Q and Q', and stores one bit, Q. There are NOR-gate and NAND-gate implementations of it, both with restriction that $SR = 0$.

- A NOR-gate gated S-R latch or a gated S-R NOR latch consists of an active-high S-R latch with S and R being replaced with the outputs of two AND gates, each of which takes E and the original input as inputs respectively.

  With present state Q, next state Q^+, and next state of Q' P, the next-state equation of a NOR-gate gated S-R latch is:

  $$Q^+ = (RE + (SE + Q)')' = (RE)'(SE + Q) = (R' + E')(SE + Q) = SR'E + R'Q + E'Q.$$

  By making $(S, R) = (1, 1)$ don't care combination, we can simplify it as:

  $$Q^+ = SE + Q(R' + E').$$

  $P = (SE+Q^+)' = (SE+SR'E+R'Q+E'Q)' = (S'+E')(R+Q')(E+Q') = S'RE+S'Q'+E'Q'.$

  $Q^{+\prime} = (SE+Q(R'+E'))' = (SE)'(Q(R'+E'))' = (S'+E')(Q'+RE) = S'RE+S'Q'+E'Q' = P.$

  By making $(S, R) = (1, 1)$ don't care combination, we can simplify it as:

  $$Q^{+\prime} = P = RE + Q'(S + E').$$

- A NAND-gate gated S-R latch or a gated S-R NAND latch consists of an active-low S-R latch with $\overline{S}$ and $\overline{R}$ being replaced with the outputs of two NAND gates, each of which takes E and the original input as inputs respectively.

  With present state Q, next state Q^+, and next state of Q' P, the next-state equation of a NAND-gate gated S-R latch is:

  $$Q^+ = ((SE)'((RE)'Q)')' = SE + (RE)'Q = SE + Q(R' + E').$$

  $P = ((RE)'Q^{+\prime})' = ((RE)'(SE+Q(R'+E')))' = RE+(SE+Q(R'+E'))' = RE+(SE)'(Q(R'+E'))' = RE$

  $Q^{+\prime} = (SE+Q(R'+E'))' = (SE)'(Q(R'+E'))' = (S'+E')(Q'+RE) = S'Q'+S'RE+E'Q = P.$

  By making $(S, R) = (1, 1)$ don't care combination, we can simplify it as:

  $$Q^{+\prime} = P = RE + Q'(S + E').$$

When $(S, R, E) = (1, 0, 1)$, it sets Q^+ to 1; when $(S, R, E) = (0, 1, 1)$, it resets Q^+ to 0; when $(S, R) = (0, 0)$ or $E = 0$, it holds present state; when $(S, R, E) = (1, 1, 1)$, which is not allowed, the outputs oscillate; when E changes from 1 to 0 while $(S, R) = (1, 1)$, which is not allowed, a race condition where both inputs to the underlying S-R latch changes from 0 to 1 and the propagation delays of the gates determine whether the latch stabilizes with Q^+ changed or not occurs.

- Another restriction of NOR-gate gated S-R latchs is when one or both of S and R go low (e.g. a glitch caused by a statc-0 hazard) near a falling edge of E, the latch may catch the unwanted 0. This is called 0's catching problem.
- Another restriction of NAND-gate gated S-R latchs is when one or both of S and R go high (e.g. a glitch caused by a statc-1 hazard) near a falling edge of E, the latch may catch the unwanted 1. This is called 1's catching problem.

The symbol of a gated S-R latch in block diagrams is an active-high S-R latch with an additional enable input line between S and R labeld En or E.

v (Gated) D Latch

A (gated) data (aka D) latch (also called transparent latch) has two inputs, data D and enable E, and two outputs, Q and Q', and stores one bit, Q.

With present state Q, next state Q^+, and next state of Q' P, the next-state equation is:

$$Q^+ = DE + QE'.$$

$$P = Q^{+\prime} = (DE + QE')' = (D' + E')(Q' + E) = D'Q' + D'E + E'Q'.$$

It is called a transparent latch because $Q = D$ when $E = 1$.

It can be implemented with a gated S-R latch with S and R being replaced with D and D' respectively, called S-R latch-based D latch, that is,

$$S = DE, \quad R = D'E,$$

or

$$\overline{S} = D' + E', \quad \overline{R} = D + E'.$$

This can be modified to use one less inverter but one more level as an active-low S-R latch with

$$\overline{S} = (DE)', \quad \overline{R} = (\overline{S}E)' = DE + E' = D + E'.$$

It can also be implemented with two transmission gates, input transmission gate T_1 and feedback transmission gate T_2, and two inverters output inverter I_1 and feedback inverter I_2, where the data input D is connected to one data terminal of T_1, the other data terminal of T_1 is connected to one data terminal of T_2, output Q, and the input of I_1, the output of I_1 is connected to the output Q' and the input of I_2, the output of I_2 is connected to the input of T_2, the enable signal of T_1 is connected to E, and the enable signal of T_2 is connected to E'

$$Q^+ = DE + QE'.$$

$$P = (DE + (Q')'E')' = (DE + QE')' = Q^{+\prime} = D'Q' + D'E + E'Q'.$$

The symbol of a D latch in block diagrams is the same as a gated S-R latch with R removed and S changed to D.

A gated D latch with inverted enable $\overline{E}$ can be implemented with a gated S-R latch with S and R being replaced with D and D' respectively and E being replaced with $\overline{E}'$, that is,

$$S = D\overline{E}', \quad R = D'\overline{E}',$$

or

$$\overline{S} = D' + \overline{E}, \quad \overline{R} = D + \overline{E}.$$

This can be modified to use one less inverter but one more level as an active-high S-R latch with

$$R = (D + \overline{E})' = D'\overline{E}', \quad S = (R + \overline{E})' = (D + \overline{E})\overline{E}' = D\overline{E}'.$$

vi (Asynchronous) J-K Latch

An (asynchronous) J-K (aka JK) latch is an improved version of the S-R latch that eliminates the forbidden state. It has two inputs, J and K, and two outputs, Q and Q', and stores one bit, Q. It consists of a gated S-R latch with S and R being renamed as J and K respectively and the enable E inputs to the gates with J and K being the other input being replaced with new Q' and Q feedback inputs respectively.

With present state Q, next state Q^+, and next state of Q' P, the next-state equation of a NOR-gate gated S-R latch, that is a J-K latch with underlying gated S-R latch being NOR-gate gated S-R latch, is:

$$Q^+ = (KQ + (JQ' + Q)')' = (KQ)'(JQ' + Q) = (K' + Q')(JQ' + Q) = JQ' + K'Q.$$

$$P = (JQ' + Q^+)' = (JQ' + K'Q)' = Q^{+\prime} = (J' + Q)(K + Q') = J'K + J'Q' + KQ.$$

With present state Q, next state Q^+, and next state of Q' P, the next-state equation of a NAND-gate gated S-R latch, that is a J-K latch with underlying gated S-R latch being NAND-gate gated S-R latch, is:

$$Q^+ = ((JQ')'((KQ)'Q)')' = JQ' + (KQ)'Q = JQ' + (K' + Q')Q = JQ' + K'Q.$$

$$P = ((KQ)'Q^+)' = (KQ)(J' + Q)(K + Q') = (J' + Q)(K + Q') = Q^{+\prime} = J'K + J'Q' + KQ.$$

When $(J, K) = (1, 0)$, it sets Q^+ to 1; when $(J, K) = (0, 1)$, it resets Q^+ to 0; when $(J, K) = (0, 0)$, it holds present state; when $(J, K) = (1, 1)$, the outputs (Q, Q') become $(1, 0)$ if originally $(0, 1)$ and $(0, 1)$ if originally $(1, 0)$, called toggle; however, if $J = K = 1$ stays longer than the propagation delay, the outputs toggle again, called the race-around problem.

The symbol of a J-K latch in block diagrams is the same as an active-high S-R latch with S and R renamed J and K respectively. Sometimes output Q' is omitted if not used.

vii Gated J-K Latch

A gated J-K latch is a J-K latch with one additional input, enable E, that is connected to both the gates J and K input to, which makes the two gates both have three inputs.

When $(J, K, E) = (1, 0, 1)$, it sets Q^+ to 1; when $(J, K, E) = (0, 1, 1)$, it resets Q^+ to 0; when $(J, K, E) = (1, 1, 1)$, the outputs toggle with race-around problem; otherwise, it holds present state.

The symbol of a gated J-K latch in block diagrams is a J-K latch with an additional enable input line between J and K labeld En.

III Flip-flops

i Edge-triggered flip-flops

An edge-triggered flip-flop has the same inputs and outputs of the gated latch of the same type but the enable input is replaced with clock input C, Clk, or CLK and for rising edge-triggered (also called positive edge-triggered or active-high) flip-flops, when clock is at a rising edge, called active (clock) edge, the flip-flop is transparent; otherwise the flip-flop is hold (also called retain); for falling edge-triggered (also called negative edge-triggered or active-low) flip-flops, when clock is at a falling edge, called active (clock) edge, the flip-flop is transparent; otherwise the flip-flop is hold (also called retain).

The symbol of a rising-edge-triggered flip-flop in block diagrams is a gated latch of the same type with enable input replaced with a small arrowhead (wedge shape) with its opening facing outwards from the rectangle and its two ends on the side of the rectangle representing the clock input.

The symbol of a falling-edge-triggered flip-flop in block diagrams is a rising-edge-triggered flip-flop of the same type with an inversion bubble outside of the rectangle at the clock input.

ii (Edge-triggered) S-R flip-flop (SRFF or SR-FF)

For active-high S-R flip-flops, at an active edge, $Q^+ = S + R'Q$ and $(S, R) = (1, 1)$ is not allowed.

For active-low S-R flip-flops, at an active edge, $Q^+ = \overline{S}' + \overline{R}Q$ and $(\overline{S}, \overline{R}) = (0, 0)$ is not allowed.

iii (Edge-triggered) D flip-flop (DFF or D-FF)

At an active edge, $Q^+ = D$.

When the context is clear, flip-flops (FFs) may be used to refer to D flip-flops (DFFs).

iv (Edge-triggered) J-K flip-flop (JKFF or JK-FF)

At an active edge, $Q^+ = JQ' + K'Q$ where when $(J, K) = (1, 1)$, the outputs toggle exactly once without race-around problem.

v (Edge-triggered) T flip-flop (TFF or T-FF)

A toggle (aka T) flip-flop has two inputs, toggle T and clock C, and two outputs, Q and Q'. At an active edge, if $T = 1$, the outputs toggle, that is, outputs (Q, Q') become $(1, 0)$ if originally $(0, 1)$ and $(0, 1)$ if originally $(1, 0)$.

With present state Q, next state Q^+, the next-state equation of a T flip-flop at an active edge is:

$$Q^+ = T'Q + TQ'.$$

It can be implemented with a J-K flip-flop with inputs J and K both connected to T.

It can also be implemented with a D flip-flop with input D connect to the output of a XOR gate with inputs T and Q, which is cheaper in cost.

The symbol of a T flip-flop in block diagrams is a D flip-flop with D replaced with T.

vi Pulse-triggered flip-flops

The term pulse-triggered flip-flop refers to the implementation that AND together clock signal and a slightly delayed version of the inversion of the clock siganl and feed it to a gated latch such that the flip-flop is transparent only on a short period after a rising edge, or AND together the inversion of a clock signal and a slightly delayed version of the clock siganl and feed it to a gated latch such that the flip-flop is transparent only on a short period after a falling edge.

vii Master-slave flip-flops

The term master-slave flip-flop refers to a particular implementation that uses two gated latches in such a way that the flip-flop outputs only change on a clock edge.

The master is the latch that takes the data inputs and the slave is the latch that takes outputs of the master as inputs and outputs the final outputs of the master-slave flip-flop.

The master-slave flip-flops need a time where the data inputs are stable before an active edge long enough for the master to latch, called setup time, and a time where the data inputs are stable after an active edge long enough for the outputs of master to transfer to the slave, called hold time. If data inputs change during setup time or hold time around an active edge, the behavior is unpredictable.

- For rising-edge-triggered flip-flops, the enable signal of the master is a slightly delayed inverted clock signal and the enable signal of the slave is the clock signal. The propagation delay of the inverter maybe used as the delay.
- For falling-edge-triggered flip-flops, the enable signal of the master is a slightly delayed clock signal and the enable signal of the slave is an inverted clock signal. A way to use propagation delay of inverter as delay is to invert the clock signal first, and feed a inverted again signal to the master's enable signal and the signal to slave's enable signal, in which the second inverter's propagation delay is the delay.

If instead that the enable signal of the slave is slightly delayed than that of the master, in an inactive edge, both latch will be enabled, which is not an intended behavior.

viii Master-slave S-R flip-flop

A master-slave S-R flip-flop can be implemented with two gated S-R latches in which Q and Q' of the master are connected to S and R of the slave respectively, S and R of the master are the

inputs of it, and Q and Q' of the slave are the outputs of it.

ix Master-slave D flip-flop

A master-slave D flip-flop can be implemented with two gated D latch in which Q of the master is connected to D of the slave, D of the master is the input of it, and Q and Q' of the slave are the outputs of it.

Alternatively, it can be implemented with a master gated D latch and a slave gated S-R latch in which Q and Q' of the master is connected to S and R of the slave, D of the master is the input of it, and Q and Q' of the slave are the outputs of it.

x Master-slave J-K flip-flop

A master-slave J-K flip-flop can be implemented with a master-slave S-R flip-flop with S and R replaced with JQ' and KQ respectively.

xi Flip-flops with asynchronous clear and preset

Some flip-flops has asynchronous inputs that change the stored state of Q at any time.

- Active-high asynchronous clear Clr sets the stored state of Q to 0 if Clr=1, called active, and does nothing if Clr=0. In block diagrams, with the side of the symbol of the flip-flop with input lines at the buttom, Clr is an input line on the left side labeled Clr.
- Active-high asynchronous clear Pre sets the stored state of Q to 1 if Pre=1, called active, and does nothing if Pre=0. In block diagrams, with the side of the symbol of the flip-flop with input lines at the buttom, Pre is an input line on the right side labeled Pre.
- Active-low asynchronous clear ClrN sets the stored state of Q to 0 if Clr=0, called active, and does nothing if Clr=1. In block diagrams, with the side of the symbol of the flip-flop with input lines at the buttom, ClrN is an input line with inversion bubble on the left side labeled ClrN or Clr.
- Active-low asynchronous clear PreN sets the stored state of Q to 1 if Pre=0, called active, and does nothing if Pre=1. In block diagrams, with the side of the symbol of the flip-flop with input lines at the buttom, PreN is an input line with inversion bubble on the right side labeled PreN or Pre.
- If both Clr (ClrN) and Pre (PreN) are active at the same time, the behavior is unpredictable.

xii Flip-flops with clock enable or chip enable

In synchronous digital systems, the flip-flops are usually driven by a common clock so that all state changes occur at the same time in response to the same clock edge. If we want a flip-flop to hold existing data during an active edge even though the data input to the flip-flops may be changing, we can use a flip-flop with clock enable (also called chip enable) (CE) input.

A flip-flop with clock enable input has an additional input CE and works as a normal flip-flop if CE=1, called enabled, and hold if CE=0, called disabled.

One method to implement it is by gating the clock, that is, replace the original clock input with the output of an AND gate with clock input and CE being the inputs. However, there are two potential problem. First, gate delays may cause the clock to arrive at some flip-flops at different times than at other flip-flops, resulting in a loss of synchronization. Second, if CE changes at the wrong time, the flip-flop may trigger due to the change in CE instead of due to the change in the clock input.

Rather than gating the clock, a better method to implement it is by replacing each original data input with the output a 2-to-1 MUX with CE being the select input, original input being the data input selected when CE=1, and 0 being the data input selected when CE=0. Because there is no gate in the clock line, this cannot cause a synchronization problem.

A flip-flop with CE is called with the original name with -CE sufficed to the original type name, e.g., D-CE flip-flop.

xiii Sequential parity checker

A sequential parity checker is a sequential circuit that receives a clock input and a serial input X of data with parity bits that is read at every clock edge and outputs $Z = 1$ if the total number of 1 inputs received is odd and $Z = 0$ total number of 1 inputs received is even.

It can be implemented with a T flip-flop, in which X is the input T and Z is the output Q.

For even parity, the state Q in the T flip-flop must be cleared to 0 before the start of a data input string. For odd parity, the state Q in the T flip-flop must be preset to 1 before the start of a data input string.

The X input must be synchronized with the clock so that it assumes its next value before the next active clock edge.

IV Analysis and Design of Sequential Circuits

i Analysis by signal tracing

The basic procedure to find the output sequence of a synchronous circuit resulting from a given input sequence is as follows:

1. Assume an initial state of the flip-flops (all flip-flops reset to 0 unless otherwise specified).
2. For the first input in the given sequence, determine the circuit output(s) and flip-flop inputs and draw on the timing chart.
3. Determine the new set of flip-flop states after the next active clock edge and draw on the timing chart.
4. Determine the output(s) that corresponds to the new states and draw on the timing chart.
5. Repeat 2, 3, and 4 for each input in the given sequence.

ii As automata

A synchronous digital circuit is a Mealy machine that transitions every active edge, called a Mealy circuit, with input alphabet being $\{0,1\}^m$ where m is the number of input wires (thus it recognize a regular language over $\{0,1\}^m$), with the variable representing elements in it denoted as X, the output alphabet being $\{0,1\}^n$ where n is the number of output wires, with the variable representing elements in it denoted as Z, states being sequences of 0's and 1's of which the length is the number of flip-flops, with the present state denoted as S and the next state denoted as S^+, next-state function, denoted as δ, given by the next-state equations of the flip-flops and the combinational logic of the flip-flop inputs as a function of present state and circuit inputs, and next-output function, denoted as λ, given by the next-state equations of the flip-flops and the combinational logic of the circuit outputs as a function of present state and circuit inputs.

When the combinational logic depends only on the state of the circuit and independent from the circuit inputs, the circuit is a Moore machine, called a Moore circuit, with next-output function, denoted as λ, given by the next-state equations of the flip-flops and the combinational logic of the circuit outputs as a function of present state.

A Mealy circuit can be specified with equations

$$S^+ = \delta(S, X), \quad Z = \lambda(S, X).$$

A Moore circuit can be specified with equations

$$S^+ = \delta(S, X), \quad Z = \lambda(S).$$

A circuit implementation with the minimal $|S|$ is called state-minimal.

iii Glitch of Mealy Circuit

After the circuit has changed state and before the inputs are changed, the outputs may temporarily assume incorrect values, called false outputs, glitches, or spikes. Thus, if any valid input is not synchronized with the clock, the best time to read the outputs is just before the active edge of the clock because the inputs must be stable at that time.

If the outputs of the circuit are fed into a second sequential circuit which uses the same clock, the false outputs will not cause any problem because the inputs to the second circuit can cause a change of state only when an active clock edge occurs.

iv Mealy and Moore implementation

For Mealy and Moore implementations of same sequential logic:

- If a Mealy implementation that is not a Moore implementation exists, Moore implementation needs more states than Mealy implementation.
- Moore implementation outputs one clock period later than Mealy implementation.
- Mealy implementation may contain glitches while Moore implementation doesn't.

v State table and (state) transition table

A (state) transition table is a one-dimensional table with three (for Moore circuit) or four (for Mealy circuit) columns that lists the corresponding next states and outputs (for Mealy circuit) in the third and fourth columns for all allowed combinations of input character in the first column and present state in the second column in rows, resembling a truth table, in which the states are represented by binary or other radix numbers corresponding to the binary sequence of the state variables.

Output column may be omitted.

Alternatively, we may list the next states given all input characters respectively given a present state in the same row with the scenario of each input character in one column and labeled in the header row.

A state table is the same as a transition table except that the states are represented by meaningless symbols assigned, often s_i or S_i.

vi State/transition/state transition diagram/graph for Moore circuit

A state/transition/state transition diagram/graph for a Moore circuit is a directed graph with

- Each vertex being a state, normally represented by circles with states in the top half and output at the state in the bottom half, i.e., [state]/[output character].
- Each directed edge (arc) being a transition, normally drawn as an arrow from present state to next state with the input character read for such transition to happen on the edge.
- The starting state is usually represented by an arrow with no origin pointing to the state, a subscript 0 in the symbol of the state (often s_0 or S_0), or not shown.

vii State/transition/state transition diagram/graph for Mealy circuit

A state/transition/state transition diagram/graph for a Mealy circuit is a directed graph with

- Each vertex being a state, normally represented by circles with states in it.
- Each directed edge (arc) being a transition, normally drawn as an arrow from present state to next state with the input character read for such transition to happen and output character of the transition on the edge separated by /, i.e., [input character]/[output character].
- The starting state is usually represented by an arrow with no origin pointing to the state, a subscript 0 in the symbol of the state (often s_0 or S_0), or not shown.

viii Incompletely specified state

A transition of state given an input character may be don't care and is represented with - and can be treated as any state when simplifying. The state, state table, and automaton are called incompletely specified when so.

An output of state for Mealy machine given an input character may be don't care and is represented with - and can be treated as any output character when simplifying. The state, state table, and automaton are called incompletely specified when so.

ix Alphanumeric state graph notation

States in a state graph are usually represented by alphanumeric symbols such as $S_0, S_1 \dots S_n$.

Input characters may also be represented in alphanumeric notation by assign alphanumeric symbols to Boolean variables of the characters and represent characters as Boolean expression of those variables, in which a Boolean expression on a transition means all characters such that the Boolean expression is true lead to that transition. When doing so, with all Boolean expressions on edges from a state s being $F_1, F_2, \dots, F_n$,

- Transition from s is deterministic iff

$$\sum_{i=1}^{n} \prod_{\substack{j=1 \\ j \neq o}}^{n} F_i F_j = 0.$$

- Transition function is defined for all combinations of s and an arbitrary input character in the input alphabet iff

$$\sum_{i=1}^{n} F_i = 1.$$

The FSM is valid as a digital circuit iff it is deterministic iff transition is deterministic for all states.

x Counter

A counter is a sequential circuit whose state changes each active edge and repeats every fixed finite number of active edges, called count cycle. The cyclic sequence of the states is called state loop. Each state in the state loop is a stage.

xi State/transition/state transition diagram/graph/cycle

A state/transition/state transition diagram/graph/cycle for a counter is a directed graph with

- Each vertex being a state.
- Each directed edge (arc) being a transition, normally drawn as an arrow from present state to next state.
- The starting state is usually represented by an arrow with no origin pointing to the state, a subscript 0 in the symbol of the state (often s_0 or S_0), or not shown.

xii Next-state Map

A next-state map is the K-map of a next-state variable as a function of the present-state variables and for Mealy circuit also input variables given by a transition table.

xiii Excitation table

An excitation table is a truth table with present and next state being input variables and flip-flop inputs being output variables that shows the minimum (that is, don't care, denoted by X, where possible) inputs that are necessary to generate a particular next state from a particular present state.

xiv Flip-flop input (K-)map

A K-map corverted from a next-state map of a flip-flop by converting each cell a flip-flop input according to the excitation table of that type of flip-flop is called a flip-flop input (K-)map.

xv D flip-flop excitation table

Q	Q^+	D
0	0	0
0	1	1
1	0	0
1	1	1

$$D = Q^+.$$

xvi T flip-flop excitation table

Q	Q^+	T
0	0	0
0	1	1
1	0	1
1	1	0

$$T = Q \oplus Q^+.$$

xvii S-R flip-flop excitation table

Q	Q^+	S	R
0	0	0	X

0	1	1	0
1	0	0	1
1	1	X	0

xviii J-K flip-flop excitation table

Q	Q^+	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

xix D input map

Conversion from a next-state map to a D input map is doing nothing.

xx T input map

Conversion from a next-state map to a T input map is complementing all 0 and 1 cells in the $Q_k = 1$ half.

xxi S and R input map

Conversion from a next-state map to a S input map is replacing all 1 cells in $Q_k = 1$ half with X cells.

Conversion from a next-state map to a R input map is replacing all 0 cells in $Q_k = 0$ half with X cells and then complementing all 0 and 1 cells.

xxii J and K input map

Conversion from a next-state map to a J input map is replacing all cells in $Q_k = 1$ half with X cells.

Conversion from a next-state map to a R input map is replacing all cells in $Q_k = 0$ half with X cells and then complementing all 0 and 1 cells.

xxiii Output functions and output maps

The output variables as functions of flip-flops outputs and for Mealy circuit also inputs of the synchronous circuit are called output functions. The K-maps of output functions are called output maps.

xxiv Analysis by state table or transition table

The following method can be used to construct a state table or a transition table:

1. Determine the flip-flop input equations and the final output equations from the circuit.
2. Derive the next-state equation for each flip-flop from its input equations.
3. Plot a next-state map for each flip-flop.
4. Combine these maps to form the table.
5. If time chart is wanted,
   (a) For the first input and starting state, read the present output and plot them.
   (b) Read the next state and plot it.
   (c) For Mealy circuit if glitches need to be plotted, go to the row in the table which corresponds to the next state and the old input column and plot the output. If the input changes several times before it assumes its correct value, the output may also change several times, plot them. This may be a false output. Otherwise, omit this step.
   (d) The input must assume its correct value before active edge. Read the next output and plot it.
   (e) Repeat step 2 to 4.

xxv Clock skew

The clock skew t_{sk} of a component i relative to another component j in a synchronous circuit is the arrival time of a clock edge to component i relative to the arrival time of a clock edge to component j.

xxvi General model for Mealy circuits

The general model of Mealy cicuits consists of k flip-flops with a common clock and a combinational subcircuit with inputs from outside of the Mealy circuit $X_1, X_2, \dots, X_m$, inputs that are flip-flops outputs $Q_1, Q_2, \dots, Q_k$, outputs as inputs for flip-flops

$$I_{11}, I_{12}, \dots, I_{1n_1}, I_{21}, I_{22}, \dots, I_{2n_2}, \dots, I_{k1}, I_{k2}, \dots, I_{kn_k}$$

with $I_{i1}, I_{i2}, \dots, I_{in_i}$ being the inputs of the ith flip-flop, and outputs to outside of the Mealy circuit $Z_1, Z_2, \dots, Z_n$.

xxvii General model for Moore circuits

The general model of Moore cicuits consists of k flip-flops with a common clock, a combinational subcircuit with inputs from outside of the Mealy circuit $X_1, X_2, \dots, X_m$ and outputs as inputs for flip-flops

$$I_{11}, I_{12}, \dots, I_{1n_1}, I_{21}, I_{22}, \dots, I_{2n_2}, \dots, I_{k1}, I_{k2}, \dots, I_{kn_k}$$

with $I_{i1}, I_{i2}, \ldots, I_{in_i}$ being the inputs of the ith flip-flop, and another combinational subcircuit with inputs that are flip-flops outputs $Q_1, Q_2, \ldots, Q_k$ and outputs to outside of the Mealy circuit $Z_1, Z_2, \ldots, Z_n$.

xxviii General model for synchronous circuits without outputs

The general model of Moore cicuits consists of k flip-flops with a common clock and a combinational subcircuit with inputs from outside of the Mealy circuit $X_1, X_2, \ldots, X_m$ and outputs as inputs for flip-flops

$$I_{11}, I_{12}, \ldots, I_{1n_1}, I_{21}, I_{22}, \ldots, I_{2n_2}, \ldots, I_{k1}, I_{k2}, \ldots, I_{kn_k}$$

with $I_{i1}, I_{i2}, \ldots, I_{in_i}$ being the inputs of the ith flip-flop.

xxix Timing constraints

TODO: check, false path

Let the clock period of a synchronous circuit be t_{clk}, and for each flip-flop i in the circuit,

- let its setup time be s_i,
- let its hold time be h_i,
- let the clock skew of it relative to the global clock signal be k_i,
- for each external input X_{ij} that any of flip-flop i's input is dependent on, let the time X_{ij} needs after an active clock edge to start being stable be x_{ij}, and the combinational circuit propagation delay from X_{ij} to the input be c_{ij}, in which if multiple inputs of flip-flop i depend on one external input, the one such that $x_{ij} + c_{ij}$ is larger is used, and
- for each flip-flop j that any of flip-flop i's input is dependent on the output(s) of flip-flop j, let the clock skew of flip-flop i relative to flip-flop j be k_{ji}, the flip-flop output propagation delay of flip-flop j to the output that flip-flop i's input is dependent on be y_{ij}, and the combinational circuit propagation delay from the output that flip-flop i's input is dependent on to flip-flop i's input be d_{ij}, in which if multiple inputs of flip-flop i depend on one flip-flop's output(s) or input(s) of flip-flop i depend on multiple outputs of flip-flop j, the one such that $y_{ij} + d_{ij}$ is larger is used,

then, the circuit is valid only if it satisfies the conditions that

$$t_{clk} \geq \max_i\{\max\{\max_j\{x_{ij} + c_{ij} + s_i - k_i\}, \max_j\{y_{ij} + d_{ij} + s_i - k_{ij}\}\},$$

and

$$\forall i : \forall j : \ y_{ij} + d_{ij} \geq k_{ij} + h_i.$$

Violation of the timing constraints are called timing violations. Violations of the first constraint are called setup violations. Violations of the second constraint are called hold violations.

xxx Output delay

When designing a sequential circuit with serial input, the number of clock cycles between the first input to the last output is called output delay.

V Register

i Register

A register can load data inputs in parallel and output stored data in parallel. A n bit register consists of n edge-triggered D flip-flops, each with D being data input and Q being data output. The flip-flops usually share common clock input and asynchronous clear and/or preset inputs. If using gated clocks, the clock is gated with a common signal called load; if using clock enable, the flip-flops share a common clock enable input called load. When load=1 at an active edge, the data inputs are loaded into the register; otherwise, the register holds present state.

The symbol of a register in block diagrams is one underlying flip-flop symbol with bus lines for data inputs and outputs, and load is labeled as Load at the clock enable input or the gated clock input.

ii Data Transfer Between Registers with Tri-state Buffers

To transfer data from a source register to a destination register, we can connect the outputs of the source register to the inputs of the destination register through tri-state buffers, and connect the enable inputs of the tri-state buffers to a common enable signal En. When $En = 1$, the tri-state buffers are enabled and the data outputs of the source register are transferred to the data inputs of the destination register; when $En = 0$, the tri-state buffers are disabled and the data inputs of the destination register are disconnected from the data outputs of the source register.

If multiple registers share a common data bus, only one register should enable its tri-state buffers at a time to avoid bus contention.

iii Data Transfer Between Registers with Multiplexers

To transfer data from source registers to a destination register, we can connect the outputs of the source registers to data inputs of a multiplexer and connect the output of the multiplexer to the data inputs of the destination register. When the select input select a source register and clock enable/load input of the destination register is active at an active edge, the data outputs of the selected source register are transferred to the data inputs of the destination register.

VI Shift Register

i Shift Register

A shift register is a sequential circuit consists of a chain of edge-triggered D flip-flops that stores data and shifts by one bit when certain control inputs are at a certain combination at an active edge, optionally allowing serial or parallel input and output.

- A shift toward least significant bit (LSB) is called a right shift.
- A shift toward most significant bit (MSB) is called a left shift.

The state variable representing the $(i+1)$-th least significant bit is denoted as Q_i, the input D of the DFF storing Q_i is denoted as D_i.

ii Serial-In Serial-Out (SISO) Shift Register

A serial-in serial-out (SISO) shift register is a type of shift register where data is shifted in and out one bit per shift. An n-bit SISO shift register consists of a chain of n edge-triggered D flip-flops. The output Q of each flip-flop is connected to the input D of the next flip-flop, with the first flip-flop in the chain receiving the serial input (SI) and the last flip-flop in the chain providing the serial output (SO) instead. The flip-flops share common clock input and clock enable/load input called shift Sh.

When $Sh = 1$ at an active edge, the register shifts a bit; otherwise, the register holds its present state.

iii Serial-In Parallel-Out (SIPO) Shift Register

A serial-in parallel-out (SIPO) shift register is a type of shift register that allows data to be retrieved in parallel. An n-bit SIPO shift register consists of an n-bit SISO register with additional $n-1$ outputs connected to the outputs Q of the flip-flops except the last one. Those outputs together with the original SO compose the parallel output of the register.

iv Parallel-In Serial-Out (PISO) Shift Register

A parallel-in serial-out (PISO) shift register is a type of shift register that allows data to be loaded in parallel. An n-bit PISO shift register consists of a chain of n edge-triggered D flip-flops, each of which associated with one 4-to-1 multiplexer (or other logically equivalent implementations). The data input D of each flip-flop is connected to the output of the associated multiplexer. The two select inputs of each multiplexer are connected to common shift enable input Sh and load enable input L. The four data inputs of each multiplexer are:

- the output of the associated flip-flop, selected when $(Sh, L) = (0, 0)$, called hold,
- the corresponding bit of the parallel input (PI), selected when $(Sh, L) = (0, 1)$, called load, and
- the output of the previous flip-flop in the chain with the first flip-flop in the chain receiving the serial input (SI) instead, selected when $(Sh, L) = (1, 0)$ or $(1, 1)$, called shift.

When $(Sh, L) = (1, 0)$ or $(1, 1)$ at an active edge, the register shifts a bit; when $(Sh, L) = (0, 1)$ at an active edge, the register loads parallel input; otherwise, the register holds its present state.

v Parallel-In Parallel-Out (PIPO) Shift Register

A parallel-in parallel-out (PIPO) shift register is a type of shift register that allows data to be loaded and retrieved in parallel. An n-bit PIPO shift register consists of an n-bit PISO register

with additional $n-1$ outputs connected to the outputs Q of the flip-flops except the last one. Those outputs together with the original SO compose the parallel output of the register.

vi Parallel-In Serial-Out (PISO) Bidirectional Shift Register

A bidirectional shift register is a type of shift register that allows data to be shifted in both directions. An n-bit parallel-in serial-out (PISO) bidirectional shift register consists of a chain of n edge-triggered D flip-flops, each of which associated with one 4-to-1 multiplexer (or other logically equivalent implementations). The data input D of each flip-flop is connected to the output of the associated multiplexer. The two select inputs of each multiplexer are connected to common shift/load input S_1 and shift direction left/right or load/hold select input S_0. The four data inputs of each multiplexer are:

- the output of the associated flip-flop, selected when $(S_1, S_0) = (0, 0)$, called hold,
- the corresponding bit of the parallel input (PI), selected when $(S_1, S_0) = (0, 1)$, called load,
- the output of the adjacent flip-flop storing the less significant bit in the chain with the flip-flop storing the least significant bit in the chain receiving the serial input (SI) instead, selected when $(S_1, S_0) = (1, 0)$, called shift right, and
- the output of the adjacent flip-flop storing the more significant bit in the chain with the flip-flop storing the most significant bit in the chain receiving the serial input (SI) instead, selected when $(S_1, S_0) = (1, 1)$, called shift left.

Two serial outputs (SO) are connected to the outputs Q of the flip-flops storing the least and most significant bits respectively.

When $(S_1, S_0) = (1, 0)$ at an active edge, the register shifts a bit right; when $(S_1, S_0) = (1, 1)$ at an active edge, the register shifts a bit left; when $(S_1, S_0) = (0, 1)$ at an active edge, the register loads parallel input; otherwise, the register holds its present state.

vii Serial-In Serial-Out (SISO) Bidirectional Shift Register

An n-bit serial-in serial-out (SISO) bidirectional shift register is an n-bit PISO register with each bit of the parallel input connected to each multiplexer as data input replaced with the output of the associated flip-flop.

When $(S_1, S_0) = (1, 0)$ at an active edge, the register shifts a bit right; when $(S_1, S_0) = (1, 1)$ at an active edge, the register shifts a bit left; otherwise, the register holds its present state.

viii Parallel-In Parallel-Out (PIPO) Bidirectional Shift Register

An n-bit parallel-in parallel-out (PIPO) bidirectional shift register is an n-bit PISO register additional $n-2$ outputs connected to the outputs Q of the flip-flops except the flip-flops storing the least and most significant bits. Those outputs together with the original two SOs compose the parallel output of the register.

ix Serial-In Parallel-Out (SIPO) Bidirectional Shift Register

An n-bit serial-in parallel-out (SIPO) bidirectional shift register is an n-bit SISO register additional $n-2$ outputs connected to the outputs Q of the flip-flops except the flip-flops storing the least and most significant bits. Those outputs together with the original two SOs compose the parallel output of the register.

x Synchronizer

An n-flip-flop synchronizer is an n-bit SISO register used to synchronize an input to a synchronous circuit with its clock, in which the original not synchronized input is inputted to the SISO register and the synchronized and delayed up to n clock periods input for the synchronous circuit is outputted from the SISO register.

A two-flip-flop synchronizer is often used.

VII Accumulator and processing unit

i Accumulator

An accumulator is a register or shift register that stores intermediate results of arithmetic and logic operations in a digital system. It is commonly used in arithmetic logic units (ALUs) of computers and digital signal processors (DSPs) to hold the results of calculations.

ii Parallel processing unit

A parallel processing unit of n cells consists of an iterative combinational circuit of n cells, each cell with i external inputs and $j < i$ external outputs, and j n-bit registers as accumulators, with external $(i-j)$ n-bit PIs and the POs of the accumulators as input of the iterative circuit, the external outputs of the iterative circuit as the PIs of the accumulators, interconnection inputs of the ends of the iterative circuit provided, and the interconnection outputs of the ends of the iterative circuit optionally catched, and processes once at each active edge when enabled.

iii Parallel adder with accumulator

Parallel adder with accumulator is an example of a parallel processing unit. An n-bit parallel adder with accumulator consists of an n-bit parallel adder and an n-bit register as accumulator. For each cell, the addend input is connected to the data output of the associated flip-flop in the accumulator, the augend input is connected to the corresponding bit of the external n-bit augend input, and the sum output is connected to the data input of the associated flip-flop in the accumulator register.

Before addition, we should let the accumulator hold the initial addend. One way is to first clear the accumulator using the asynchronous clear inputs on the flip-flops, and then load the addend via the augend input. Another way is to add a multiplexer before the data input of each flip-flop in the accumulator register to select between the external data input and the sum output of the adder.

iv Serial processing unit

A serial processing unit is a logical equivalent of a parallel processing unit, but instead of processing multiple bits (or words) simultaneously, it processes data bit by bit (or word by word) over multiple clock cycles. For a parallel processing unit whose iterative combinational subcircuit have s cells, each has n external inputs, m external outputs, and k interconnection inputs (thus k interconnection outputs) all toward the same direction (i.e., ith cell to $(i+1)$ th cell) that processes repeatedly t times, the logically equivalent serial processing unit consists of m s-bit shift registers as accumulators, $(n-m)$ external $s \times t$-bit SIs (sometimes from s-bit SO shift registers whose SOs are connected back to SIs), a cell of the iterative circuit, a k-bit register, and a control state machine that receives a start input St and provides a shift output Sh (both assumed active-high here, inverting gives an active-low one) to all the accumulators and external SIs, with the external SIs and the SOs of the accumulators as input of the cell, the external outputs of the cell as the SIs of the accumulators, the interconnection outputs as inputs of the k-bit register, the outputs of the k-bit register as the interconnection inputs, and the control state machine being: (assumed Mealy here)

- If St won't stay 1 after $s \times t$ active edges, or such 1 should be treated as a new start: with $s \times t$ states, $S_0, S_1, \ldots, S_{s \times t-1}$, where S_0 is the starting state and the transitions are: (in present state input/output→ next state with x denoting don't care)

$$S_0 0/0 \to S_0,$$

$$S_0 1/1 \to S_1,$$

$$S_j x/1 \to S_{j+1}, \quad j \in \mathbb{N} \wedge j < s \times t - 1,$$

$$S_{s \times t-1} x/1 \to S_0.$$

- If St may stay 1 after $s \times t$ active edges, and should 1 not be treated as a new start: with $s \times t + 1$ states, $S_0, S_1, \ldots, S_{s \times t}$, where S_0 is the starting state, $S_{s \times t}$ is the stop state, and the transitions are: (in present state input/output→ next state with x denoting don't care)

$$S_0 0/0 \to S_0,$$

$$S_0 1/1 \to S_1,$$

$$S_j x/1 \to S_{j+1}, \quad j \in \mathbb{N} \wedge j < s \times t,$$

$$S_{s \times t} 1/0 \to S_{s \times t},$$

$$S_{s \times t} 0/0 \to S_0.$$

v Serial adder with accumulator

Serial adder with accumulator is an example of a serial processing unit. An n-bit serial adder with accumulator consists of one full adder, two n-bit PIPO shift registers with one of them as accumulator, a D flip-flop, and a control state machine of n (if St won't stay 1 after n active edges, or such 1 should be treated as a new start) or $(n+1)$ (if St may stay 1 after n active edges, and should 1 not be treated as a new start) states. The addend and augend is loaded into the

two shift registers first, with LSB in the last FF. The two SO is connected to the addend and augend input of the FA, the carry-out is connected to the input of the DFF, the output Q of the DFF is connected to the carry-in of the FA, the sum output of the FA is connected to the SI of the accumulator, and the SO of the augend register is sometimes connected to the SI of the augend register. After n active edges, the addend register is loaded with the LSB to the nth bit of the sum with LSB in the last FF, and the sum output of the FA is the $(n+1)$th bit of the sum.

VIII Synchronous Counter

i Synchronous Counter

A synchronous counter is a counter consists of flip-flops sharing the same clock input.

ii Shift Register Counter

A shift register counter is a type of synchronous counter consists of a shift register with some or all the outputs Q of the flip-flops being connected to the inputs of a combinational circuit with one output whose output is connected to D_0. A shift register counter of n flip-flops is called n-bit or n-stage. For an n-bit shift register counter, D_0 as an n-variable Boolean function of $Q_0, Q_1, \dots, Q_{n-1}$ is called feedback function of the shift register counter.

iii Ring Counter

A ring counter is a type of shift register counter consists of a shift register with the output Q of the last flip-flop in the chain connected to the input D of the first flip-flop in the chain. The bits stored circulate around the register. The count cycle of an n-bit ring counter is n.

iv Johnson counter, Twisted Ring Counter, or Mod-2n Counter

A Johnson counter, twisted ring counter, or mod-2n counter is a type of shift register counter consists of a shift register with the output Q' of the last flip-flop in the chain connected to the input D of the first flip-flop in the chain. The legal states of a Johnson counter are those consist of at most one contiguous 0 bits block and at most one contiguous 1 bits block. For an n-bit Johnson counter, number of legal states is $2n$, and the count cycle is $2n$ if starting with a legal state.

v Binary Up Counter

An n-bit synchronous binary up counter consists of n T flip-flops sharing a common clock input signal with the one storing the $(k+1)$-th least significant bit labeled with subscript $_k$. The data input T_0 is connected to a constant 1 source, and the data input T_k for $k \in \mathbb{N}$ is connected to

$$\prod_{i=0}^{k-1} Q_i,$$

which is implemented using AND gates as

$$T_1 = Q_0$$

$$T_k = T_{k-1}Q_{k-1}, \quad k \in \mathbb{N} \wedge 1 < k < n.$$

Its count cycle is 2^n and its state is a binary number incrementing from 0 to 2^n one-by-one per cycle.

vi Down Counter

An n-bit synchronous down counter consists of n T flip-flops sharing a common clock input signal with the one storing the $(k+1)$-th least significant bit labeled with subscript $_k$. The data input T_0 is connected to a constant 0 source, and the data input T_k for $k \in \mathbb{N}$ is connected to

$$\prod_{i=0}^{k-1} Q'_i,$$

which is implemented using AND gates as

$$T_1 = Q'_0$$

$$T_k = T_{k-1}Q'_{k-1}, \quad k \in \mathbb{N} \wedge 1 < k < n.$$

Its count cycle is 2^n and its state is a binary number decrementing from 2^n to 0 one-by-one per cycle.

vii Up/Down (U/D) Counter

An n-bit synchronous up/down (U/D) counter with separate up U and down D inputs consists of n T flip-flops sharing a common clock input signal with the one storing the $(k+1)$-th least significant bit labeled with subscript $_k$. The data input T_k is connected to

$$U \prod_{i=0}^{k-1} Q_i + D \prod_{i=0}^{k-1} Q'_i, \quad k \in \mathbb{N}_0 \wedge k < n$$

which is implemented using AND and OR gates or other logically equivalent combinational circuit (e.g., NAND gates) as

$$U_0 = U$$

$$U_k = U_{k-1}Q_{k-1}, \quad k \in \mathbb{N} \wedge k < n$$

$$D_0 = D$$

$$D_k = D_{k-1}Q'_{k-1}, \quad k \in \mathbb{N} \wedge k < n$$

$$T_k = U_k + D_k, \quad k \in \mathbb{N} \wedge k < n.$$

An n-bit synchronous up/down (U/D) counter with up/down input M indicating up when $M = 1$ and down when $M = 0$ consists of n T flip-flops optionally with clock enable CE to hold when CE=0 sharing a common clock input signal with the one storing the $(k+1)$-th least significant bit labeled with subscript $_k$. The data input T_0 is connected to a constant 1 source, and the data input T_k for $k \in \mathbb{N}$ is connected to

$$M \prod_{i=0}^{k-1} Q_i + M' \prod_{i=0}^{k-1} Q'_i, \quad k \in \mathbb{N}_0 \wedge k < n$$

which is implemented using AND and OR gates or other logically equivalent combinational circuit (e.g., NAND gates) as

$$U_1 = M$$

$$U_k = U_{k-1}Q_{k-1}, \quad k \in \mathbb{N} \wedge 1 < k < n$$

$$D_1 = M'$$

$$D_k = D_{k-1}Q'_{k-1}, \quad k \in \mathbb{N} \wedge 1 < k < n$$

$$T_k = U_k + D_k, \quad k \in \mathbb{N} \wedge 1 < k < n.$$

viii Loadable Counter

A counter that consists of T flip-flops mentioned above can be adjusted to be loada le from a parallel input (PI) with an associated 2-to-1 multiplexer for each T flip-flop and a common load input as select input for all multiplexers. The original input T to each flip-flop is connected instead to the data input of the associated multiplexer which is selected when load=0, the output of each multiplexer is connected to the input T of the associated flip-flop, and the other data input to each multiplexer, which is selected when load=1, is connected to the corresponding bit of the parallel input.

When load=1 at an active edge, the loadable counter loads parallel input; when load=0 at an active edge, the loadable counter works as the underlying counter; otherwise, the loadable counter holds its present state.

ix Design of General Synchronous Counter with T Flip-Flops

To design a synchronous counter consists of n T flip-flops given arbitrary transition table of m rows of distinct present states Q_k's and corresponding next states Q_k^+'s specified, where the other $2^n - m$ states are don't care states, the following procedure can be used:

1. For each Q_k^+, draw a next-state map.
2. Convert the next-state maps to T input maps.
3. Realize all inputs T with combination circuits with a subset of all outputs Q and Q', constant 1 and 0 sources, and all external control inputs as inputs.

x Design of General Synchronous Counter with D Flip-Flops

To design a synchronous counter consists of n D flip-flops given arbitrary transition table of m rows of distinct present states Q_k's and corresponding next states Q_k^+'s specified, where the other $2^n - m$ states are don't care states, the following procedure can be used:

1. For each Q_k^+, draw a next-state map.
2. Convert the next-state maps to D input maps.
3. Realize all inputs D with combination circuits with a subset of all outputs Q and Q', constant 1 and 0 sources, and all external control inputs as inputs.

xi Design of General Synchronous Counter with S-R Flip-Flops

To design a synchronous counter consists of n S-R flip-flops given arbitrary transition table of m rows of distinct present states Q_k's and corresponding next states Q_k^+'s specified, where the other $2^n - m$ states are don't care states, the following procedure can be used:

1. For each Q_k^+, draw a next-state map.
2. Convert the next-state maps to S and R input maps.
3. Realize all inputs S and R with combination circuits with a subset of all outputs Q and Q', constant 1 and 0 sources, and all external control inputs as inputs.

xii Design of General Synchronous Counter with J-K Flip-Flops

To design a synchronous counter consists of n J-K flip-flops given arbitrary transition table of m rows of distinct present states Q_k's and corresponding next states Q_k^+'s specified, where the other $2^n - m$ states are don't care states, the following procedure can be used:

1. For each Q_k^+, draw a next-state map.
2. Convert the next-state maps to J and K input maps.
3. Realize all inputs J and K with combination circuits with a subset of all outputs Q and Q', constant 1 and 0 sources, and all external control inputs as inputs.

IX Linear Feedback Shift Register (LFSR)

A linear feedback shift register (LFSR) is a shift register counter whose feedback function is linear, or a circuit that is logically equivalent to a such shift register counter. LFSR is commonly used to generate pseudo-random numbers and pseudo-noise sequences in modern computer system.

An n-bit LFSR is characterized by its feedback function f. Let $T \subseteq \{0, 1, \dots, n-1\}$ be such that

$$f = \bigoplus_{i \in T} Q_i.$$

Each feedback from Q_i for $i \in T$ is called a tap. The feedback polynomial, characteristic polynomial, or connection polynomial $p(x)$ is defined as

$$p(x) = 1 + \sum_{i \in T} x^{i+1}.$$

TODO: definition check, $\mathbb{F}_2$ abstract algebra interpretation of feedback polynomial

The initial state of an LFSR is called seed.

For each $n \in \mathbb{N}$, the maximal count cycle of n-bit LFSRs is $2^n - 1$, in which all possible states except the state where all bits are zero are included. An n-bit LFSR has count cycle $2^n - 1$ iff its feedback polynomial is primitive over $\mathbb{F}_2$.

TODO: primitive, irreducible, Primitive trinomials in abstract algebra

TODO: The state graph of an n-bit LFSR (with $2^n - 1$ vertices) consists of only disjoint cycles and loops, and the state where all bits are zero is always with a loop.

TODO: Fibonacci LFSRs, Galois LFSRs, Xorshift LFSRs

TODO end

X Asynchronous counter or ripple counter

i Asynchronous counter or ripple counter

An asynchronous counter or a ripple counter is a counter consists of flip-flops not sharing the same clock input.

ii Asynchronous down and up counter or frequency divider

An n-bit (also called mod-2^n) asynchronous down counter consists of n FFs such that $Q_{n-1}Q_{n-2}\dots Q_0$ as an unsigned number decrements by one every active clock edge. A rising-edge asynchronous down counter consists of n rising-edge triggered TFFs with all inputs T connected to constant 1 source, Clk_0 connected to the clock signal, and Clk_{k+1} connected to Q_k for all $k \in \mathbb{N}_0 \wedge k < n$. A falling-edge asynchronous down counter consists of n falling-edge triggered TFFs with all inputs T connected to constant 1 source, Clk_0 connected to the clock signal, and Clk_{k+1} connected to Q'_k for all $k \in \mathbb{N}_0 \wedge k < n$.

An n-bit (also called mod-2^n) asynchronous up counter consists of n FFs such that $Q_{n-1}Q_{n-2}\dots Q_0$ as an unsigned number increments by one every active clock edge. A rising-edge asynchronous up counter consists of n rising-edge triggered TFFs with all inputs T connected to constant 1 source, Clk_0 connected to the clock signal, and Clk_{k+1} connected to Q'_k for all $k \in \mathbb{N}_0 \wedge k < n$. A falling-edge asynchronous up counter consists of n falling-edge triggered TFFs with all inputs T connected to constant 1 source, Clk_0 connected to the clock signal, and Clk_{k+1} connected to Q_k for all $k \in \mathbb{N}_0 \wedge k < n$.

An n-bit asynchronous up or down counter is also a divided-by-2^n frequency divider, in which Q_{n-1} and Q'_{n-1} are clock signals whose periods being 2^n times the period of the Clk and duty cycles being 2^{n-1} times the period of the Clk.

XI Sequence Detector

i Definition

A sequence detector is a Mealy circuit with a serial input $X \in \{0, 1\}$ is read and an output Z that when the string of the last n characters in the input word read is a prescribed string w, it outputs 1; otherwise, it outputs 0. Let $w = a_1a_2\dots a_n$, $w_i = a_1a_2\dots a_i$ for all $1 \le i \le n$, and $w_0 = \varepsilon$ (empty string).

Initial state can be loaded such that the detector works as if a input string is already read at the beginning.

Types:

- **Overlapping sliding window sequence detector**: Detector is never reset. This is the default one.
- **Non-overlapping sliding window sequence detector**: Detector is reset when $Z = 1$ occurs.
- **Disjoint window sequence detector**: The detector is reset every n input characters after the first occurrence of $Z = 1$.

ii Design with SIPO shift register

- **Moore circuit implementation:** Use an n-bit SIPO shift register to read X and a combinational circuit whose inputs are the parallel outputs of the shift register that outputs 1 if the shift register stores w now and 0 otherwise to output Z.

  The output becomes valid after the next clock edge, but the output is stable for the entire clock period (independent of input glitches).

- **Mealy circuit implementation:** Use an $(n-1)$-bit SIPO shift register to read X and a combinational circuit whose inputs are the parallel outputs of the shift register and the input X that outputs 1 if the shift register stores w_{n-1} now and $X = a_n$ and 0 otherwise to output Z.

  The output is valid since the final input bit arrives (one clock earlier than Moore), but the output is combinational with respect to X, so it may glitch if X is not stable or combinational subcircuit hasn't stabilized. One less flip-flop is used than Moore circuit implementation.

For the detector to work as if a input string v is already read at the beginning, load v into the shift register at the beginning with parallel or serial input.

To prevent $Z = 1$ to occur before the $(n-m)$th input character is read where $m = 0$ for those without initial state and the length of initial state v for those with initial state v, we can AND the original output Z together with the output C of a combinational subcircuit that outputs 1 if a mod-n counter is at the nth state and 0 otherwise as the new output Z, with the clock enable signal of the counter connected to C', and the original state of the counter being the $(m+1)$th state.

For non-overlapping sliding window sequence detector, connect a clear signal for all flip-flops to Z.

For disjoint window sequence detector, we can AND the original output Z together with the output C of a combinational subcircuit that outputs 1 if a mod-n counter with parallel input is at the nth state and 0 otherwise as the new output Z, with the clock enable signal of the counter connected to C' and the parallel input connected to a constant source of first state of the counter read iff $Z = 1$.

iii Design as finite-state machine (FSM) performing Knuth–Morris–Pratt (KMP) algorithm

1. Find the number of states and flip-flops needed. Let the set of all states be S. Let the number of flip-flops be k. Each state S_i corresponds to the scenario that the string of the last i input characters is w_i, and for any $j > i$, the string of the last j input characters is not w_j.

   - **Moore circuit implementation:** We need $n+1$ states,

   $$S = \{S_0, S_1, \dots, S_n\},$$

   $$k \geq \lceil \log_2(n+1) \rceil.$$

   - **Mealy circuit implementation:** We need n states,

   $$S = \{S_0, S_1, \dots, S_{n-1}\},$$

   $$k \geq \lceil \log_2(n) \rceil.$$

2. Let transition function be $\delta : S \times \{0,1\} \to S$. We have $\delta(S_i, a_{i+1})$ defined as

   $$\delta(S_i, a_{i+1}) = S_{i+1}$$

   - **Moore circuit implementation:** for any $0 \leq i \leq n-1$.
   - **Mealy circuit implementation:** for any $0 \leq i \leq n-2$.

3. Assign a unique k-bit binary number whose each digit corresponds to the state variable of a flip-flop to each state.

4. Compose output:

   - **Moore circuit implementation:** Use a combinational circuit whose inputs are the outputs of the flip-flops that outputs 1 if the present state is S_n and 0 otherwise. The output function $\lambda : S \to \{0,1\}$ is defined as

   $$\lambda(s) = \begin{cases} 1, & s = S_n \\ 0, & s \in S \setminus \{S_n\} \end{cases}.$$

   - **Mealy circuit implementation:** Use a combinational circuit whose inputs are the outputs of the flip-flops and X that outputs 1 if the present state is S_{n-1} AND $X = a_n$ and 0 otherwise. The output function $\lambda : S \times \{0,1\} \to \{0,1\}$ is defined as

   $$\lambda(s, a) = \begin{cases} 1, & s = S_{n-1} \wedge a = a_n \\ 0, & s \in S \setminus \{S_{n-1}\} \vee a \neq a_n \end{cases}.$$

5. Define the **prefix/failure function/link** $\pi : S \to S$ such that $\pi(S_0) = S_0$, and for any $S_i \in S \setminus \{S_0\}$ with $S_p = \pi(S_i)$, it satisfies that $p < i$, that w_p equals to the string of the last p characters of w_i, and that for any $q \in (p, i)$, w_q doesn't equal to the string of the last q characters of w_i. However for

- **Non-overlapping sliding window sequence detector Moore circuit implementation:**

$$\pi(S_n) = S_0$$

instead.

6. We have $\delta(S_i, a'_{i+1})$ defined recursively as

$$\delta(S_i, a'_{i+1}) = \delta(\pi(S_i), a_{i+1})$$

for any $0 \leq i \leq n-1$, and

$$\delta(S_n, a) = \delta(\pi(S_n), a)$$

for any $a \in \{0, 1\}$.

7. Draw a state graph that:

- **Moore circuit implementation:**

$$s/\lambda(s) \xrightarrow{a} \delta(s)/\lambda(\delta(s))$$

for all $s \in S$ and $a \in \{0, 1\}$, equivalently,

$$S_i/0 \xrightarrow{a_{i+1}} S_{i+1}/0, \quad 0 \leq i \leq n-2,$$

$$S_{n-1}/0 \xrightarrow{a_n} S_n/1,$$

$$S_i/0 \xrightarrow{a'_{i+1}} \delta(\pi(S_i), a'_{i+1})/0, \quad 0 \leq i \leq n-1,$$

$$S_n/1 \xrightarrow{a} \delta(\pi(S_n), a), \quad a \in \{0, 1\}.$$

- **Mealy circuit implementation:**

$$s \xrightarrow{a/\lambda(s,a)} \delta(s)$$

for all $s \in S$ and $a \in \{0, 1\}$, equivalently,

$$S_i \xrightarrow{a_{i+1}/0} S_{i+1}, \quad 0 \leq i \leq n-2,$$

$$S_{n-1} \xrightarrow{a_n/1} \delta(\pi(S_{n-1}), a_n),$$

$$S_i \xrightarrow{a'_{i+1}/0} \delta(\pi(S_i), a'_{i+1}), \quad 0 \leq i \leq n-1.$$

Note that all δ and λ are computed in the process, since in hardware you never implement it recursively.

8. Draw a state table based on the state graph.
9. Translate the state table to transition table based on the encoding.
10. Plot the next-state maps for the flip-flops and the K-map for the output Z.
11. Convert next-state maps to flip-flop input maps.

12. Implement the flip-flop inputs and output Z with combination logic as functions of input character variable and present state variable.
13. For disjoint window sequence detector, we can AND the original output Z together with the output C of a combinational subcircuit that outputs 1 if a mod-n counter with parallel input is at the nth state and 0 otherwise as the new output Z, with the clock enable signal of the counter connected to C' and the parallel input connected to a constant source of first state of the counter read iff $Z = 1$.

iv Trie

In computer science, a trie, also known as a digital tree or prefix tree, is a tree data structure where each edge stores a character and each node has a one-to-one correspondence to the string of the characters of the edges from root to it concatenated one by one, where the root corresponds to ε (empty string).

Given a set of words $W = \{w_1, w_2, \dots, w_n\}$. Let $w_i = a_{i_1} a_{i_2} \dots a_{i_{|w_i|}}$, $w_{i_j} = a_{i_1} a_{i_2} \dots a_{i_j}$ for all i_j such that $j \in \mathbb{N} \wedge j \leq |w_i|$, $w_{i_0} = \varepsilon$ (empty string), and $W_i = \{w_{i_j} \mid j \in \mathbb{N}_0 \wedge j \leq |w_i|\}$, for all $i \in \mathbb{N} \wedge i \leq n$. Let the set $P = \bigcup_{i=1}^{n} W_i$. Note that if w_{i_j} and w_{k_l} are the same, they are the same element of P. The trie of $w_1, w_2, \dots, w_n$ is a trie such that the nodes is a bijection of P and the nodes correspond to $w_1, w_2, \dots, w_n$ respectively labeled ”end-of-word”. Note that in the trie, a node labeled ”end-of-word” is not necessarily a leaf node (a node without child), but a leaf node must be labeled ”end-of-word”.

v Sequence detector detecting multiple sequences

A sequence detector detecting multiple sequences $w_1, w_2, \dots, w_n$ outputs the same as the outputs of the n sequence detectors detecting $w_1, w_2, \dots, w_n$ respectively ORed together, or outputs a sequence of n bits corresponding to the outputs of the n sequence detectors detecting $w_1, w_2, \dots, w_n$ respectively.

It can be implemented the n sequence detectors detecting $w_1, w_2, \dots, w_n$ respectively, with each of them implemented with SIPO register or as FSM performing KMP algorithm. However, this method wastes flip-flops.

We can implement it with FSM performing Aho-Corasick algorithm.

1. Construct a trie of $w_1, w_2, \dots, w_n$.
2. Find the number of states and flip-flops needed. Let the set of all states be S. Each state has a one-to-one correspondence to a node. Let the number of flip-flops be k.

$$k \geq \lceil \log_2(|S|) \rceil.$$

   - **Moore circuit implementation:** The states in S is bijection of the nodes.
   - **Mealy circuit implementation:** The states in S is bijection of the nodes with children.

XII Line code converter

i Line code

In telecommunications, a line code is a pattern of voltage, current, or photons used to represent serial digital data transmitted.

ii Clock recovery

Clock recovery is a process in serial communication used to extract timing information from a stream of serial data being sent in order to accurately determine payload sequence without separate clock information.

iii DC balance

In telecommunications, DC balance refers to the property of a transmitted signal that the DC component approaches when the length of the signal approaches infinity for any signal pattern.

iv NRZ (non-return-to-zero) code

NRZ code is a line code of serial binary data that represent bit ’1’ with high voltage for the entire bit period and bit ’0’ with low voltage for the entire bit period.

It requires minimal bandwidth and has poor clock recovery and no DC balance. Often used internally within chips and on circuit boards where distances are short.

v NRZI (non-return-to-zero-inverted) code

NRZI code is a line code of serial binary data that represent bit ’1’ with a transition in voltage (high-to-low or low-to-high) at the beginning of the bit period and bit ’0’ with no transition for the entire bit period.

It requires minimal bandwidth and has poor clock recovery for long 0s and no DC balance. Used in the Universal Serial Bus (USB).

vi RZ (return-to-zero) code

RZ code is a line code of serial binary data that represent bit ’1’ with a pulse that goes high at the beginning of the bit period and return 0 before the end of the bit period and bit ’0’ with low voltage for the entire bit period.

It requires more bandwidth than NRZ code and has good clock recovery for 1s and no DC balance.

vii Manchester code

Manchester code is a line code of serial binary data that represent bit ’1’ with a high-to-low transition in voltage in the middle of the bit period and bit ’0’ with a low-to-high transition in voltage in the middle of the bit period.

It requires twice the bandwidth of NRZ code and has excellent clock recovery and DC balance. Used in ethernet (10BASE-T) and many RFID tags (e.g., NFC).

viii Mealy NRZ to NRZI and NRZI to NRZ converter

A D flip-flop with the same active edges occurrence as input signal that store last bit and a XOR gate that XOR input signal and flip-flop output as output signal.

ix Moore NRZ to NRZI and NRZI to NRZ converter

A two-bit SIPO shift register with the same active edges occurrence as input signal that store last two bits and a XOR gate that XOR the two flip-flops' output as output signal.

x Mealy NRZ to RZ converter

An AND gate that AND the input signal and a clock signal of the same frequency as that of the input signal that is at rising edge at the beginning of every bit.

xi Moore NRZ to RZ converter

A D flip-flop with the same active edges occurrence as input signal that store last bit and an AND gate that AND the flip-flop output and a clock signal of the same frequency as that of the input signal that is at rising edge at the beginning of every bit.

xii Moore RZ to NRZ converter

A D flip-flop with the same active edges occurrence as input signal that store last bit sampled for 1 bits between beginning of bit periods and high-to-low transitions whose output is the output signal.

xiii Mealy NRZ to Manchester converter

A FSM with clock signal being of two times the frequency of the clock of the input signal of three states S_0, S_1, S_2 with S_0 being the starting state and state graph

$$S_0 \xrightarrow{0/0} S_1 \xrightarrow{0/1} S_0 \xrightarrow{1/1} S_2 \xrightarrow{1/0} S_0.$$

It can be implemented with two D flip-flops with input D_0D_1 and output Q_0Q_1. Let $S_0 = 00, S_1 = 01, S_2 = 10$, input signal X, and output signal Z.

Q_0	Q_1	X	Q_0^+	Q_1^+	Z
0	0	0	0	1	0
0	0	1	1	0	1
0	1	0	0	0	1
1	0	1	0	0	0

Q_0^+	00	01	11	10
0	0	0	X	X
1	1	X	X	0

$$D_0 = Q_0' X.$$

Q_1^+	00	01	11	10
0	1	0	X	X
1	0	X	X	0

$$D_1 = Q_1' X'.$$

Q_1^+	00	01	11	10
0	0	1	X	X
1	1	X	X	0

$$Z = Q_0' X + Q_1.$$

xiv Moore NRZ to Manchester converter

A FSM with clock signal being of two times the frequency of the clock of the input signal of four states S_0, S_1, S_2, S_3 with S_0 or S_2 being the starting state and state graph

$$S_0/0 \xrightarrow{0} S_1/0 \xrightarrow{0} S_2/1 \xrightarrow{0} S_1/0,$$

$$S_2/1 \xrightarrow{1} S_3/1 \xrightarrow{1} S_0/0 \xrightarrow{1} S_3/1.$$

It can be implemented with two D flip-flops with input $D_0 D_1$ and output $Q_0 Q_1$. Let $S_0 = 00, S_1 = 01, S_2 = 10, S_3 = 11$, input signal X, and output signal Z.

Q_0	Q_1	X	Q_0^+	Q_1^+
0	0	0	0	1
0	0	1	1	1
0	1	0	1	0
1	0	0	0	1

1	0	1	1	1
1	1	1	0	0

Q_0^+	00	01	11	10
0	0	1	X	0
1	1	X	0	1

$$D_0 = Q_1 \oplus X.$$

Q_1^+	00	01	11	10
0	1	0	X	1
1	1	X	0	1

$$D_1 = Q_1'.$$
$$Z = Q_0.$$

XIII Elimination of Redundant States

i Reachability

A state is reachable iff it is mapped to by transition function δ for any $x \in D_\delta$.

ii Distinguishable

Two states in a deterministic finite state transducer are distinguishable iff there exists an input string such that the output string starting from the two states are different.

Two states in a deterministic finite state transducer are k-distinguishable iff there exists an input string of length k such that the output string starting from the two states are different.

iii Equivalent states

Two completely specified states p and q of a Mealy machine are equivalent, denoted $p \equiv q$, iff for any input X,

$$\delta(p, X) \equiv \delta(q, X) \wedge \lambda(p, X) = \lambda(q, X).$$

Two completely specified states p and q of a Moore machine are equivalent, denoted $p \equiv q$, iff for any input X,

$$\delta(p, X) \equiv \delta(q, X),$$

and

$$\lambda(p) = \lambda(q).$$

Equivalence of states is reflexive, symmetric, and transitive.

Two completely specified states are distinguishable iff they can't be equivalent in any assignment of legal equivalent states.

If two states are equivalent, we can merge them to one.

A completely specified deterministic finite-state transducer is minimal iff no state is unreachable and every two distinct states are distinguishable.

We can minimize a completely specified deterministic finite-state transducer by maximizing the number of pairs of equivalent states, eliminating all equivalences, and eliminating all unreachable states.

iv Implication chart or implication table

Implication chart, also called implication table, is a method to minimize completely specified deterministic finite-state transducer.

First eliminate all unreachable states. Let the set of all remaining states be $S = \{S_0, S_1, \dots, S_n\}$ and the input alphabet be $\Sigma = \{X_0, X_1, \dots, X_m\}$.

1. The implication chart is a lower-triangle table with entries a_{ij} for all integers $i, j \in \mathbb{N}_0 \wedge n \geq i > j$. In each entry, a pair of state (s, t) is denoted as $s - t$, and pairs of states are denoted row by row.
2. Fill each entry a_{ij} with
   - $\times$ (means distinguishable) iff for Mealy machine, there exists $X_k \in \Sigma$ such that $\lambda(S_j, X_k) \neq \lambda(S_i, X_k)$, and for Moore machine, $\lambda(S_j) \neq \lambda(S_i)$,
   - pairs of states $(\delta(S_j, X_k), \delta(S_i, X_k))$ for all $X_k \in \Sigma$ such that

     $$\delta(S_j, X_k) \neq \delta(S_i, X_k) \wedge \{\delta(S_j, X_k), \delta(S_i, X_k)\} \neq \{S_i, S_j\},$$

     otherwise.
3. For each entry a_{ij} that has not been filled with $\times$, fill it with $\times$ iff any pair of states (S_p, S_q) in it is such that $a_{\max(p,q)\min(p,q)}$ has been filled with $\times$. Repeatedly iterate through all entries that has not been filled with $\times$ until no entry is filled with $\times$ in an iteration.
4. For each entry a_{ij} that has not been filled with $\times$, $S_j \equiv S_i$. For all equivalence classes under $\equiv$, merge all states in it to one.

v Compatible states

Two possibly incompletely specified states p and q of a Mealy machine are compatible, denoted $p \sim q$, iff for any input X such that both $\delta(p, X)$ and $\delta(q, X)$ are defined,

$$\delta(p, X) \sim \delta(q, X),$$

and for any input X such that both $\lambda(p, X)$ and $\lambda(q, X)$ are defined,

$$\lambda(p, X) = \lambda(q, X).$$

Two possibly incompletely specified states p and q of a incompletely specified Moore machine are compatible, denoted $p \sim q$, iff for any input X such that both $\delta(p, X)$ and $\delta(q, X)$ are defined,

$$\delta(p, X) \sim \delta(q, X),$$

and if both $\lambda(p)$ and $\lambda(q)$ are defined,

$$\lambda(p) = \lambda(q).$$

Compatibility of states is reflexive and symmetric but not transitive.

Two states are compatible if they are equivalent.

Two states are distinguishable iff they can't be compatible in any assignment of legal compatible states.

If two states are compatible, we can merge them to one.

A incompletely specified deterministic finite-state transducer is minimal iff no state is unreachable and every two distinct states are distinguishable.

vi Merger table or merger chart

Merger table, also called merger chart, implication chart, or implication table, is a method to minimize incompletely specified deterministic finite-state transducer.

First eliminate all unreachable states. Let the set of all remaining states be $S = \{S_0, S_1, \dots, S_n\}$ and the input alphabet be $\Sigma = \{X_0, X_1, \dots, X_m\}$.

1. The merger table is a lower-triangle table with entries a_{ij} for all integers $i, j \in \mathbb{N}_0 \wedge n \geq i > j$. In each entry, a pair of state (s, t) is denoted as $s - t$, and pairs of states are denoted row by row.
2. Fill each entry a_{ij} with
   - $\times$ (means distinguishable) iff for Mealy machine, there exists $X_k \in \Sigma$ such that $\lambda(S_j, X_k) \neq \lambda(S_i, X_k)$, and for Moore machine, $\lambda(S_j) \neq \lambda(S_i)$,
   - pairs of states $(\delta(S_j, X_k), \delta(S_i, X_k))$ for all $X_k \in \Sigma$ such that both $\delta(S_j, X_k)$ and $\delta(S_i, X_k)$ are specified and that

     $$\delta(S_j, X_k) \neq \delta(S_i, X_k) \wedge \{\delta(S_j, X_k), \delta(S_i, X_k)\} \neq \{S_i, S_j\},$$

     otherwise.
3. For each entry a_{ij} that has not been filled with $\times$, fill it with $\times$ iff any pair of states (S_p, S_q) in it is such that $a_{\max(p,q)\min(p,q)}$ has been filled with $\times$. Repeatedly iterate through all entries that has not been filled with $\times$ until no entry is filled with $\times$ in an iteration.

4. For each entry a_{ij} that has not been filled with $\times$, $S_j \sim S_i$. Draw a undirected graph with each state being a vertex and an edge exists between two vertices iff the two states are compatible, called compatibility graph.
5. Find all maximal cliques of the graph using the Bron–Kerbosch algorithm.
6. Draw a undirected graph with each maximal clique found in the last step being a vertex and an edge exists between two vertices iff the two cliques are disjoint.
7. Find all maximal cliques of the graph using the Bron–Kerbosch algorithm.
8. Find the maximal clique with the maximal sum of the number of vertices in the clique of compatibility graph corresponding to each vertex in the clique. Merge all vertices in each of the clique of compatibility graph corresponding to each vertex in that clique to one.

vii Equivalent deterministic finite-state transducers

Two (completely or incompletely specified) deterministic finite-state transducers M_1, M_2 with same input alphabets Σ and output alphabets Γ, each with set of all states S, T with $S \cap T = \varnothing$, transition functions δ_1, δ_2, and output function λ_1, λ_2, are equivalent, iff there exists a bijection $f : S \to T$ such that with function

$$\forall (s, X) \in D_{\delta_1} : \delta_1(s, X) = \delta_2(f(s), X),$$

$$\forall (t, X) \in D_{\delta_2} : \delta_2(t, X) = \delta_1(f^{-1}(t), X),$$

and,

- for Mealy machine

$$\forall (s, X) \in D_{\lambda_1} : \lambda_1(s, X) = \lambda_2(f(s), X),$$

$$\forall (t, X) \in D_{\lambda_2} : \lambda_2(t, X) = \lambda_1(f^{-1}(t), X),$$

- for Moore machine

$$\forall s \in D_{\lambda_1} : \lambda_1(s) = \lambda_2(f(s)),$$

$$\forall t \in D_{\lambda_2} : \lambda_2(t) = \lambda_1(f^{-1}(t)).$$

viii Finding equivalent deterministic finite-state transducers with implication chart or pair chart

First minimize the two (completely or incompletely specified) deterministic finite-state transducers respectively. Let the set of all remaining states of the two automata be S and T. If $|S| \neq |T|$, the two automata are not equivalent. Let $S = \{S_0, S_1, \dots, S_n\}$ and $T = \{T_0, T_1, \dots, T_n\}$, and the input alphabet of both automata be $\Sigma = \{X_0, X_1, \dots, X_m\}$.

1. The implication chart, also called implication table or pair chart, is a table with entries a_{ij} for all integers $i, j \in \mathbb{N}_0 \wedge n \geq i, j$.
2. Fill each entry a_{ij} with

- × (means distinguishable) iff for Mealy machine, there exists $X_k \in \Sigma$ such that $\lambda(S_i, X_k) \neq \lambda(T_j, X_k)$, and for Moore machine, $\lambda(S_i) \neq \lambda(T_j)$,
- pairs of states $(\delta(S_i, X_k), \delta(T_j, X_k))$ for all $X_k \in \Sigma$ such that

$$\delta(S_i, X_k) \neq \delta(T_j, X_k) \wedge (\delta(S_i, X_k), \delta(T_j, X_k)) \neq (S_i, T_j),$$

  otherwise.

3. For each entry a_{ij} that has not been filled with ×, fill it with × iff any pair of states (S_p, T_q) in it is such that a_{pq} has been filled with ×. Repeatedly iterate through all entries that has not been filled with × until no entry is filled with × in an iteration.
4. Define a bipartite graph with two partitions S and T with, an edge between $S_i \in S$ and $T_j \in T$ exists iff a_{ij} is not filled with ×. The two automata are equivalent iff the bipartite graph has a perfect matching.

ix Conversion between Moore and Mealy machine

- Moore machine to Mealy machine: A Moore machine is already a Mealy machine. However, a minimal Moore machine may not be a minimal Mealy machine, minimization by merging equivalent or compatible states may be required.
- Mealy machine to Moore machine: For each state, let there be n distinct outputs, split it to n states of the distinct outputs. A Moore machine converted from a minimal Mealy machine is always minimal.

XIV State Assignment

A unique binary number whose each digit corresponds to the state variable of a flip-flop is assigned to each state.

i Cost

After the number of states of the automaton has been reduced, the next step in realizing is to assign flip-flop state variables to correspond to the states of the automaton The cost of the logic required to realize a sequential circuit is strongly dependent on the way this state assignment is made.

ii Equivalent state assignments

Two state assignments are called equivalent iff they are of equal cost.

iii Interchanging digits

If two state assignments are such that one can be obtained from the other by interchanging two digits for all states, the two state assignments are equivalent, since the interchanging is equivalent to interchanging the order of two flip-flops.

iv Symmetrical flip-flop

A flip-flop is symmetrical iff complementing the bit stored in that flip-flop for all states in a state assignment results in a state assignment of equal cost. S-R, J-K, and T flip-flops are symmetrical, while D flip-flops aren't.

v Trial-and-error method

The trial-and-error method is a method to find the state assignment(s) of the lowest cost as follows

1. For a n-state DFSM, $k = \lceil \log_2 n \rceil$ flip-flops is required.
2. Find equivalent state assignments by interchanging digits, complementing bits for symmetrical flip-flops, etc. and eliminate one of them for every two equivalent state assignments found.
3. Iterate through the remaining state assignments and find the minimum cost solution(s).

vi Assignment map method

The trial-and-error method is not practical in most cases. The assignment map method involves trying to choose a state assignment which will place the 1's (or 0's) on the flip-flop input maps and output maps in adjacent cells so that the corresponding terms can be combined. This method does not apply to all problems, and even when applicable, it does not guarantee a minimum cost solution.

Assignments for two states are said to be adjacent if they differ in only one state variable. The following guidelines are used to find the state assignments that place the 1's (or 0's) on the flip-flop input maps (guidelines 1 and 2) and output maps (guideline 3) in adjacent cells:

1. States which have the same next state for a given input should be given adjacent assignments.
2. States which are the next states of the same state should be given adjacent assignments.
3. States which have the same output for Mealy circuit for a given input should be given adjacent assignments.
4. When above guidelines require that three states be adjacent, these states should be assigned such that for the three two-combinations from the three states, only one of them is such that the two states are not adjacent.
5. As the number of state variables increases, the cost of not satisfying guidelines 1 and 2 grows approximately exponentially.
6. As the number of output variables increases, the cost of not satisfying guideline 3 grows approximately linearly.

When using this guidelines, we write down all pairs states which should be given adjacent assignments according to the guidelines and then draw assignment maps. An assignment

map is a Karnaugh map in which the rows and columns are combinations of state variables and each cell is a unique assignment for a state to be filled in. When filling in the map, the starting state should be put in the cell where all state variables are 0 to simplify reset wiring since nothing is to be gained by trying to put the starting state in different squares on the map because the same number of adjacencies can be found no matter where you put the starting state. Fill the map and try to satisfy as many of these adjacencies as possible. A state that is equivalent to a tried state need not be tried. Such equivalence can be found by interchanging digits, complementing bits for symmetrical flip-flops, etc. An amount of trial and error may be required to fill in the map so that the maximum number of desired state adjacencies is obtained.

vii One-hot and one-cold state assignment

When a one-hot (or one-cold) state assignment is used,

- The minimal SOP (or POS) form of next-state equation for each flip-flop is such that each product term (or sum term) contains exactly one state variable.
- The minimal SOP (or POS) form of output function for each output is such that each product term (or sum term) contains exactly one state variable.

XV Design of Sequential Circuits Using ROMs, PLAs, CPLDs, and FGPAs

i Design of Sequential Circuits Using ROMs

The next-state and output combinational subcircuits of a sequential circuit can be realized using ROMs (read-only memories). The state of the circuit can then be stored in a register of D flip-flops and fed back to the input of the ROM.

- A Mealy sequential circuit with m inputs, n outputs, and k state variables can be realized using k D flip-flops and a ROM with $m + k$ inputs (2^{m+k} words) and $n + k$ outputs.
- A Moore sequential circuit with m inputs, n outputs, and k state variables can be realized using k D flip-flops, a next-state ROM with $m + k$ inputs (2^{m+k} words) and k outputs, and an output ROM with k inputs (2^k words) and n outputs.

Use of D flip-flops is preferable because use of two-input flip-flops would require increasing the number of outputs from the ROM. The fact that the D flip-flop input equations could require more gates is of no consequence because the size of the ROM depends only on the number of inputs and outputs and not on the complexity of the equations being realized. For this reason, the state assignment is also of no consequence.

ii Design of Sequential Circuits Using PLAs

The next-state and output combinational subcircuits of a sequential circuit can be realized using PLAs (programmable logic arrays). However, in the case of PLAs, the state assignment may be important because the use of a good state assignment can reduce the required number of product terms and, hence, reduce the required size of the PLA.

iii Design of Sequential Circuits Using CPLDs and FGPAs

When designing with CPLDs and FPGAs, we should keep in mind that flip-flops in logic cells are there whether we use them or not. This means that it may not be important to minimize the number of flip-flops used in the design. Instead, we should try to reduce the total number of logic cells used and the interconnections between cells since the propagation delay is longer when several cells are cascaded to realize a function. Using a one-hot or one-cold encoding for state assignment may help to accomplish this.

When designing with CPLDs or FPGAs, you should try both an assignment with a minimum number of state variables and a one-hot (or one-cold) assignment to see which one leads to a design with the smallest number of logic cells and/or fewest interconnections among cells.

XVI Simulation and Testing

Simulation of sequential circuits is similar to the simulation of combinational circuits. However, the delays associated with the individual logic elements must be taken into account and modeled during simulation. The simulator output usually includes timing diagrams which show the times at which different signals in the circuit change. The simplest method is to assume that each element has one unit of delay. If a more detailed timing analysis is required, each logic element may be assigned nominal delay values, which are usually provided by the device manufacturer on the specification sheets with minimum, typical, and maximum delay values.

Testing of sequential circuits is generally more difficult than testing combinational circuits. If the flip-flop outputs can be observed, then the state table can be verified directly on a row-by-row basis with a simulator or in lab as follows for each present state in the table:

1. Use the preset and clear inputs to set the flip-flop states to the present state.
2. For a Moore machine, check to see that the output is correct. For a Mealy machine, check to see that the output is correct for each input combination.
3. For each input combination, clock the circuit and check to see that the next state of the flip-flops is correct.

In the cases where a sequential circuit is implemented as part of an integrated circuit, only the inputs and outputs are available at the IC pins, and observing the state of the internal flip-flops is Impossible, test must be done by applying input sequences to the circuit and observing the output sequences. The set of test sequences must traverse all arcs on the state graph, but this is typically not a sufficient test.

XVII Multiplier

i General

Multiplication of two unsigned binary numbers $A = A_{n-1}A_{n-2}\dots A_0$ and $B = B_{n-1}B_{n-2}\dots B_0$ involves two steps, computing the n partial products $P_i = AB_i << i$ where $<< i$ means shifting

i bits towards MSB for $i \in \mathbb{Z} \cap [0, n-1]$, and computing multi-operand addition $\sum_{i=0}^{n-1} P_i$ to get the final result.

Bit heap is the heap of partial products one partial product per row, where each column is also called weight and the number of partial products containing that column is called height. For the plain n partial products described above, column i (starting at 0) has $i+1$ bits for $i \in \mathbb{Z} \cap [0, n-1]$ and $2n-i-1$ bits for $i \in \mathbb{Z} \cap [n, 2n-2]$, and the maximum height is n at column $n-1$.

A full adder can reduce 3 bits of the same column to one bit of the same column and one bit of the next column. A half adder can reduce 2 bits of the same column to one bit of the same column and one bit of the next column.

ii Shift-add multiplier

Use a shifter and an accumulator to sum each partial product one by one, trading off speed for area.

iii Array multiplier

Sum partial products with carry-save adder until the number of operands left is two, and use an adder to sum the two. The number of stage is $O(n)$.

iv Wallace (tree/reduction) multiplier

In each stage of Wallace multiplier, for each column of bit heap with height h in parallel, $\left\lfloor \frac{h}{3} \right\rfloor$ full adders are used, and if $h \mod 3 = 2$, a half adder is also used, until all columns have height ≤ 2. And then an adder is used to get the final result. The number of stage in total is $O(\log n)$.

v Dadda (tree/reduction) multiplier

Dadda defined a sequence $\langle d_k \rangle$:

$$d_1 = 2,$$
$$d_{k+1} = \lfloor \frac{3}{2} d_k \rfloor, \quad k \in \mathbb{N},$$

that is, $2, 3, 4, 6, 9, 13, 19, \ldots$.

In each stage of Dadda multiplier, choose the largest k such that $d_k \leq$ the height of the highest column, for each column of bit heap with height h in parallel but with carry bit from last column in current stage (if any) counted when calculating full and half adders used, $\left\lfloor \frac{h-d_k}{2} \right\rfloor$ full adders are used, and if $(h-d_k) \mod 2 = 1$, a half adder is also used, until all columns have height ≤ 2. And then an adder is used to get the final result. The number of stage in total is $O(\log n)$, and the gates and area used is less than that of Wallace multiplier except some small n.

vi Booth's multiplication algorithm or Booth encoding

Booth's multiplication algorithm or Booth encoding is a technique to reduce the number of partial products and makes them work for two's complement numbers (but not for unsigned numbers without tweak or constraint). Those partial products are then fed in to Wallace or Dadda tree.

Suppose there are two two's complement signed numbers, multiplicand $A = A_{n-1}A_{n-2}\dots A_0$ and multiplier $B = B_{n-1}B_{n-2}\dots B_0$, B_{-1} is defined as 0.

Consider a run of ones $B_sB_{s-1}\dots B_t$, called starting at bit t and ending at bit $s+1$. It contributes $B^{s+1} - B^t$. If bit i is 0 and bit $i-1$ is 1, there is a end of a run of ones at bit i, contributing 2^i. If bit i is 1 and bit $i-1$ is 0, there is a start of a run of ones at bit i, contributing -2^i. If bits i and $i-1$ are the same, there is no start or end of a run of ones at bit i, contributing 0. Thus bit i contributes $(B_{i-1} - B_i) * 2^i$. If B_0 is 1, there is a start of a run of ones at bit 0. This is why B_{-1} is defined as 0. Consider last bit, bit $n-1$. If bit $n-1$ is 0 and bit $n-2$ is 1, there is a end of a run of ones at bit $n-1$, contributing 2^{n-1}. If bit $n-1$ is 1 and bit $n-2$ is 0, the number is negative, contributing -2^{n-1}. If bit $n-1$ is 0 and bit $n-2$ is 0, there is no end of a run of ones at bit $n-1$, contributing 0. If bit $n-1$ is 1 and bit $n-2$ is 1, the number is negative, contributing -2^{n-1}, and there is an end of run of ones at bit $n-1$, contributing 2^{n-1}, thus contributing 0 in total. Therefore, the same rule can be applied directly to the last bit.

Consider $B_{k(i+1)-1}\dots B_{ki-1}$. $B_{k(i+1)-1}\dots B_{ki}$ contributes

$$
\begin{aligned}
m &= \sum_{j=ki}^{k(i+1)-1} (B_{j-1} - B_j) * 2^j \\
&= \sum_{j=ki}^{k(i+1)-1} B_{j-1} * 2^j - \sum_{j=ki}^{k(i+1)-1} B_j * 2^j \\
&= B_{ki-1} * 2^{ki} + \sum_{j=ki}^{k(i+1)-2} B_j * 2^{j+1} - \sum_{j=ki}^{k(i+1)-2} B_j * 2^j - B_{k(i+1)-1} * 2^{k(i+1)-1} \\
&= B_{ki-1} * 2^{ki} + \sum_{j=ki}^{k(i+1)-2} B_j * 2^j - B_{k(i+1)-1} * 2^{k(i+1)-1} \\
&= \left(B_{ki-1} + \sum_{j=0}^{k-2} B_{ki+j} * 2^j - B_{k(i+1)-1} * 2^{k-1} \right) * 2^{ki},
\end{aligned}
$$

which has range $[-2^{k-1}, 2^{k-1}] * 2^{ki}$.

When using Booth radix-2^k encoding, assuming n is a multiple of k, the algorithm scans the multiplier $k+1$ bits at a time with one bit overlapping with last scan and starting from $B_{k-1}\dots B_{-1}$ to $B_{n-1}\dots B_{n-k-1}$, which are $\frac{n}{k}$ windows in total and $B_{k(i+1)-1}\dots B_{ki-1}$ in each scan $i \in \mathbb{Z} \cap \left[0, \frac{n}{k} - 1\right]$, and computes partial product $m * A * 2^{ki}$ where $m = B_{ki-1} + \sum_{j=0}^{k-2} B_{ki+j} * 2^j - B_{k(i+1)-1} * 2^{k-1}$ is retrieved from a lookup table given $B_{k(i+1)-1}\dots B_{ki-1}$.

The 2^z multiples of A for $z \in \mathbb{N}$ can be computed by shifting A z bits towards MSB. Only radix-2

and radix-4 has m that ranges only on exponents of 2, which are $\{-1, 0, 1\}$ and $\{-2, -1, 0, 1, 2\}$ respectively. And radix-4 has fewer windows than radix-2 and is the most used one.

If only n bits is needed, we can simply discard upper bits from partial products.

Now consider unsigned A, B. The result is the result of two's complement plus $A * B_{n-1} * 2^n$, which can be computed by adding it as another partial product, or, if only n bits is needed, be ignored since it's in the overflowing bits.

XVIII Divider

i General

Division of two unsigned binary number, dividend $N = N_{n-1}N_{n-2} \dots N_0$ and divisor $D = D_{n-1}D_{n-2} \dots D_0$, gives two unsigned binary number, quotient $Q = Q_{n-1}Q_{n-2} \dots Q_0$ and remainder $R = R_{n-1}R_{n-2} \dots R_0$ such that

$$N = D * Q + R$$

and

$$0 \leq R < D.$$

Three n-bit registers, remainder register R holding partial remainder and finally holding remainder, quotient register Q initially holding dividend N, and divisor register D holding divisor, and one subtractor are used. Treat R, Q as one combined register.

ii Restoring Divider

In each iteration,

1. Shift R, Q left by one bit.
2. Assign $R - D$ to R.
3. Check R in 2's complement.
    - If $R > 0$, set the LSB of Q to 1.
    - If $R < 0$, assign $R + D$ to R, called restoring.

Iterate n times to obtain the final result.

iii Non-restoring Divider

Let a flag bit F exist and be initially set to 0. In each iteration,

1. Shift R, Q left by one bit.
2. Check F.
    - If $F = 0$, assign $R - D$ to R.
    - If $F = 1$, assign $R + D$ to R.
3. Check R in 2's complement.

- If $R > 0$, set the LSB of Q to 1 and F to 0.
- If $R < 0$, set F to 1.

Iterate n times.

- If $F = 0$, do nothing.
- If $F = 1$, assign $R + D$ to R.

Now we've obtained the final result.

iv SRT Divider

The algorithm is named after D. W. Sweeney of IBM, James E. Robertson of University of Illinois, and K. D. Tocher of Imperial College London. They all developed the algorithm independently at approximately the same time (published in February 1957, September 1958, and January 1958 respectively).

For a radix-2^k SRT divider, assuming n is a multiple of k, an extra $\frac{n}{k}$-cell storage Q' where each cell can store an integer in $[-2^{k-1}, 2^{k-1}]$ is used to store intermediate quotient in that form.

In each ith iteration,

1. Shift R, Q left by k bits.
2. Use a LUT with a number of leading bits of D and a number of leading bits of R as inputs to determine an integer $q_i \in [-2^{k-1}, 2^{k-1}]$ such that $\left|R - q_i D\right| < D$.
3. Assign $R - q_i D$ to R and store q_i to the ith cell of Q'.

Iterate $\frac{n}{k}$ times.

Let a flag bit F exist and be initially set to 0. If $R < 0$, assign $R + D$ to R and F to 1.

Translate Q' to final quotient Q:

$$Q = \sum_{i=1}^{\frac{n}{k}} q_i * 2^{i-1} - F.$$

Now we've obtained the final result.

A fast Q' storage is carry-save storage. It consists of an n-bit register Q_s, an $n + 1$-bit register Q_c, and an n-bit carry-save adder. Q_s, Q_c are set to zero initially. q_i are represented in $k + 1$ bit two's complement. Each ith iteration,

1. Q_s, Q_c are both shifted left k bits.
2. The carry-save adder takes Q_s, Q_c, q_i as inputs and Q_s, Q_c as sum and carry outputs.

And the final quotient Q is

$$Q = Q_s + Q_c - F.$$

XIX Arithmetic Logic Unit (ALU)

An arithmetic logic unit (ALU) is a combinational digital circuit that performs operations on integer binary numbers. The common operations include:

- Unsigned and two's complement arithmetic operations: addition, subtraction, increment, decrement, negation.
- Bitwise logical operations: AND, OR, XOR.
- Bit shift operations: Logical right shift (SRL) (0 as new MSB), logical left shift (SLL) (0 as new LSB), arithmetic right shift (SRA) (old MSB as new MSB), arithmetic left shift (SRL) (old LSB as new LSB), rotate right (old LSB as new MSB), rotate left (old MSB as new LSB).

Comparison operations are typically done by checking flags of subtraction.

Multiplication and division usually have their seperate units.

XX State Machine Chart (SM Chart)

State machine chart, SM chart, or algorithmic state machine (ASM) chart is a type of flowchart that is equivalent to a state graph. The componenets of an SM chart include:

- State box: A rectangular box that contains a state name, and may contain an output list written after a slash after or the state name. A state code may be placed outside the box at the top.
- Decision box: A diamond-shaped box that contains condition, a Boolean expression that is evaluated to true or false, and two branches, true and false.
- Conditional output box: A curved-ended rectangular box that contains conditional output list, the outputs that depend on both the state and the inputs.

A state graph for a sequential FSM can be converted to and from an equivalent SM chart. The Moore outputs become output list in a state box. The Mealy inputs become conditional output box. Derivation and realization of SM charts are similar to those of state graphs.